



VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

BRNO UNIVERSITY OF TECHNOLOGY

FAKULTA ELEKTROTECHNIKY

A KOMUNIKAČNÍCH TECHNOLOGIÍ

FACULTY OF ELECTRICAL ENGINEERING AND COMMUNICATION

ÚSTAV TELEKOMUNIKACÍ

DEPARTMENT OF TELECOMMUNICATIONS

METODY DETEKCE STAVU ODEPŘENÍ SLUŽBY

DENIAL OF SERVICE DETECTION METHODS

BAKALÁŘSKÁ PRÁCE

BACHELOR'S THESIS

AUTOR PRÁCE

AUTHOR

Milan Horský

VEDOUCÍ PRÁCE

SUPERVISOR

Ing. Marek Sikora

BRNO 2024

Bakalářská práce

bakalářský studijní program **Informační bezpečnost**

Ústav telekomunikací

Student: Milan Horský

ID: 230258

Ročník: 3

Akademický rok: 2023/24

NÁZEV TÉMATU:

Metody detekce stavu odepření služby

POKYNY PRO VYPRACOVÁNÍ:

Cílem této bakalářské práce je vytvořit metodiku sloužící k jednoznačnému popisu a detekci stavu odepření služby (Denial of Service - DoS) na webovém serveru a tuto metodu následně implementovat v experimentální síti. V teoretické části práce prostudujte a zhodnoťte metody k určení stavu DoS. Zeměťte se na možnosti detekce na straně aplikačního serveru a na straně sítě. Na straně serveru se zaměřte na závislosti mezi hardwarovými a softwarovými zdroji. Na straně sítě diskutujte důvody falešně pozitivních výsledků detekce (např. rate limiting, geo blokace, firewall, preventivní konfigurace serveru atd.) a pokuste se vytvořené metody o tyto poznatky vylepšit. V praktické části práce vytvořte softwarové nástroje, které budou získávat data a generovat přehledné statistiky o vytíženosti jednotlivých HW (CPU, RAM, HDD) a SW (sokety, pakety, data) zdrojů ve formě webové prezentace a pokuste se přehledně zobrazit jejich závislosti. Vytvořené nástroje otestujte proti vybraným DoS útokům (5 záplavových a 5 pomalých) a diskutujte zachycené jevy vzhledem k detekci DoS. Na základě teoretické studie a získaných dat navrhnete optimální metodiku pro detekci stavu DoS na webových serverech a aplikujte/implementujte ji ve vaší experimentální síti.

DOPORUČENÁ LITERATURA:

Dle pokynů vedoucího práce.

Termín zadání: 5.2.2024

Termín odevzdání: 28.5.2024

Vedoucí práce: Ing. Marek Sikora

doc. Ing. Jan Hajný, Ph.D.
předseda rady studijního programu

ÚPOZORNĚNÍ:

Autor bakalářské práce nesmí při vytváření bakalářské práce porušit autorská práva třetích osob, zejména nesmí zasahovat nedovoleným způsobem do cizích autorských práv osobnostních a musí si být plně vědom následků porušení ustanovení § 11 a následujících autorského zákona č. 121/2000 Sb., včetně možných trestněprávních důsledků vyplývajících z ustanovení části druhé, hlavy VI. díl 4 Trestního zákoníku č.40/2009 Sb.

ABSTRAKT

Tato bakalářská práce se zabývá metodikou pro detekci a popis stavu odepření služby (Denial of Service - DoS) na webovém serveru. Cílem práce je vyvinout a implementovat metody, které umožní efektivní detekci DoS útoků v experimentální síti. Teoretická část se zaměřuje na analýzu různých typů DoS útoků, včetně jejich detekce na straně aplikačního serveru a sítě, a diskutuje možné důvody falešně pozitivních výsledků. V praktické části byly vytvořeny softwarové nástroje pro monitorování a analýzu serverových a síťových zdrojů, které byly následně testovány proti různým typům DoS útoků. Výsledkem práce je navržená metodika, která byla ověřena a implementována v experimentální síti, což poskytuje užitečné nástroje pro zvýšení bezpečnosti a stability webových serverů.

KLÍČOVÁ SLOVA

Detekce, DoS, Záplavové útoky, Logické útoky, Python, Node.js, PostgreSQL, Grafy, Serverové metriky, Virtualizace, Proxmox, Testování

ABSTRACT

This bachelor thesis deals with a methodology for detecting and describing the Denial of Service (DoS) condition on a web server. The aim of the thesis is to develop and implement methods that enable effective detection of DoS attacks in an experimental network. The theoretical part focuses on the analysis of different types of DoS attacks, including their detection on the application server and network side, and discusses possible reasons for false positives. In the practical part, software tools for monitoring and analyzing server and network resources were developed and then tested against different types of DoS attacks. As a result of the work, the proposed methodology has been validated and implemented in an experimental network, which provides useful tools for improving the security and stability of web servers.

KEYWORDS

Detection, DoS, Flood Attacks, Logic Attacks, Python, Node.js, PostgreSQL, Graphs, Server Metrics, Virtualization, Proxmox, Testing

ROZŠÍŘENÝ ABSTRAKT

Internet se stal nezbytnou součástí moderní společnosti, propojující miliardy zařízení po celém světě a umožňující okamžitý přístup k informacím, komunikaci a obchodním transakcím. S nárůstem internetového provozu a závislosti na online službách se zvyšuje také potřeba zabezpečení těchto služeb před různými typy kybernetických útoků. Mezi nejběžnější patří útoky typu Denial of Service (DoS), jejichž cílem je vyřadit z provozu cílové servery nebo sítě tím, že je zaplaví nadměrným množstvím požadavků nebo zneužívají specifické vlastnosti síťových protokolů.

DoS útoky představují vážnou hrozbu pro organizace všech velikostí, protože mohou způsobit významné finanční ztráty, narušit důvěru zákazníků a poškodit pověst společnosti. Tyto útoky se mohou lišit svou povahou a způsobem provedení, od jednoduchých záplavových útoků, které se snaží zahlcením přenosové kapacity vyřadit cílový server, po sofistikovanější pomalé útoky, které využívají nedostatky v implementaci síťových protokolů a aplikačních logik.

Sofistikovanost a rozmanitost DoS útoků vyžaduje pokročilé metody detekce a prevence. Úspěšná obrana proti těmto útokům zahrnuje kombinaci technických opatření, jako jsou firewally, systémy pro detekci a prevenci narušení (IDS/IPS) a různé metody monitorování síťového provozu. Klíčovým faktorem je schopnost včasné identifikace útoku, což umožňuje rychlou reakci a minimalizaci dopadů na provozní dostupnost služeb.

Cílem této bakalářské práce je vytvořit metodiku pro jednoznačný popis a detekci stavu odepření služby (DoS) na webovém serveru a tuto metodu následně otestovat v experimentální síti. V teoretické části se zaměřuji na analýzu různých typů DoS útoků a na možnosti jejich detekce jak na straně aplikačního serveru, tak na straně sítě. Zvláštní pozornost je věnována vztahům mezi hardwarovými a softwarovými zdroji a diskusi o falešně pozitivních výsledcích detekce.

Teoretická část práce začíná úvodem do základních konceptů síťové komunikace, včetně protokolů TCP/IP a modelu ISO/OSI. Protokolová sada TCP/IP je základním stavebním kamenem internetu, kde jednotlivé vrstvy mají specifické funkce: aplikační vrstva, transportní vrstva, síťová vrstva a fyzická vrstva. Model ISO/OSI poskytuje strukturovanější pohled na síťovou komunikaci, rozdělenou do sedmi vrstev, což usnadňuje pochopení a analýzu různých typů síťových útoků.

Následuje detailní analýza různých typů DoS útoků, včetně jejich charakteristik a mechanismů. DoS útoky jsou klasifikovány do několika kategorií, jako jsou záplavové útoky (např. SYN Flood, ICMP Flood) a pomalé útoky (např. Slowloris, Slow Read). Záplavové útoky se zaměřují na zahlcení cílového systému obrovským množstvím dat a je proti nim velmi těžké bojovat, neboť pokud má útočník výkonnější připojení k internetu, je schopen generovat větší provoz, než vůbec může obět přijmout. Pomalé útoky zase zneužívají specifické nedostatky v protokolech nebo

aplikacích k udržování otevřených spojení a vyčerpávání zdrojů serveru. Tyto útoky jsou zase nebezpečné v tom, že na první pohled vypadají jako běžný provoz a k jejich realizaci není potřeba nijak velké síťové spojení.

Praktická část práce se zaměřuje na vývoj softwarové aplikace, která slouží k monitorování a vizualizaci nasbíraných dat o vytíženosti serverových a síťových zdrojů. Rozděluje se na 2 velké skupiny, servery pro sběr dat ze serveru a z prostředí sítě a webová aplikace s databází pro sběr a ukládání dat. Skripty byly vytvořeny v jazyce Python a využívají technologie knihovny jako je `psutil` pro načítání dat a `threading` pro práci s více vlákny. Webová aplikace je pak vytvořena v prostředí Node.js s databází PostgreSQL. Tato aplikace umožňuje uživatelům sledovat různé metriky, jako jsou zatížení CPU, využití RAM, disková kapacita, počet socketů, počet přenesených paketů a objem přenesených dat. Aplikace byla navržena tak, aby poskytovala přehledné a intuitivní rozhraní pro sledování historie a aktuálního stavu serverových a síťových zdrojů. Aplikace také umožňuje sbírání dat do více proměnných a je tak možné ukládat průběhy útoků a následně je mezi sebou porovnávat. Uživatelé mohou také exportovat nasbíraná data ve formátu CSV, výsledný soubor pak obsahuje přesně ty data, které uživatel měl zobrazené. Tento nástroj umožňuje uživatelům provádět vlastní analýzu dat a identifikovat potenciální DoS útoky na základě nasbíraných metrik.

Další část studie se dostává k samotné metodice detekce DoS útoků, včetně jejich výhod a nevýhod. Dočteme se zde o metodách používaných v IDS a IPS systémech, které se proti DoS útokům běžně používají. Diskutuje se zde také o problematice falešně pozitivních výsledků, které mohou být způsobeny například rate limitingem, geo blokadami, firewalley a preventivní konfigurací serverů. Tyto aspekty jsou důležité pro pochopení, jaké výzvy přináší detekce DoS útoků v reálném prostředí a jak lze tyto výzvy překonat.

Následně jsou navrženy a posány 2 metody pro detekci DoS útoků. Testování těchto navržených metodik probíhalo v experimentálním virtualizovaném prostředí, kde byly simulovány různé typy DoS útoků, včetně pěti záplavových a pěti pomalých útoků. Testy zahrnovaly útoky jako SYN Flood, ICMP Flood, Slowloris a Slow Read. Výsledky testování ukázaly, že metody jsou schopny spolehlivě detekovat většinu testovaných útoků a poskytla cenné poznatky o chování serveru pod útokem.

Navržená metodika pro detekci DoS útoků byla úspěšně otestována v experimentální síti. Aplikace pro monitorování a vizualizaci dat se ukázala být účinným nástrojem pro sledování různých metrik a poskytování přehledu o stavu serveru a sítě. Nicméně během testování bylo identifikováno několik omezení, například neschopnost získávat statistiky z Apache serveru během intenzivních útoků, což snižuje použitelnost aplikace v těchto situacích. Přesto aplikace poskytuje užitečné nástroje pro monitorování a analýzu dalších metrik, což zvyšuje její celkovou hodnotu. Dalším

zlepšením by mohl být přechod na soketovou komunikaci pro přenos dat mezi skripty a backendem a mezi backendem a frontendem, což by zlepšilo plynulost datového toku.

Navržené metody dokáží spolehlivě detekovat různé typy útoků, ačkoli jejich použití v reálném provozu by bylo složitější a vyžadovalo by pokročilejší techniky, jako je strojové učení. Celkově tato práce přináší konkrétní řešení pro detekci DoS útoků a otevírá prostor pro další výzkum a zdokonalování v této oblasti. Testovaná metodika a nástroje poskytují cenné prostředky pro zvýšení bezpečnosti a stability webových serverů a přispívají k lepšímu porozumění problematice DoS útoků. Tato práce tak nejen přináší konkrétní výsledky a praktická řešení, ale také otevírá nové možnosti pro budoucí výzkum a vývoj v oblasti detekce a prevence DoS útoků.

HORSKÝ, Milan. *Metody detekce stavu odepření služby*. Bakalářská práce. Brno: Vysoké učení technické v Brně, Fakulta elektrotechniky a komunikačních technologií, Ústav telekomunikací, 2024. Vedoucí práce: Ing. Marek Sikora

Prohlášení autora o původnosti díla

Jméno a příjmení autora: Milan Horský
VUT ID autora: 230258
Typ práce: Bakalářská práce
Akademický rok: 2023/24
Téma závěrečné práce: Metody detekce stavu odepření služby

Prohlašuji, že svou závěrečnou práci jsem vypracoval samostatně pod vedením vedoucí/ho závěrečné práce a s použitím odborné literatury a dalších informačních zdrojů, které jsou všechny citovány v práci a uvedeny v seznamu literatury na konci práce.

Jako autor uvedené závěrečné práce dále prohlašuji, že v souvislosti s vytvořením této závěrečné práce jsem neporušil autorská práva třetích osob, zejména jsem nezasáhl nedovoleným způsobem do cizích autorských práv osobnostních a/nebo majetkových a jsem si plně vědom následků porušení ustanovení § 11 a následujících autorského zákona č. 121/2000 Sb., o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon), ve znění pozdějších předpisů, včetně možných trestněprávních důsledků vyplývajících z ustanovení části druhé, hlavy VI. díl 4 Trestního zákoníku č. 40/2009 Sb.

Brno
.....
podpis autora*

*Autor podepisuje pouze v tištěné verzi.

PODĚKOVÁNÍ

Rád bych poděkoval vedoucímu bakalářské práce panu Ing. Marku Sikorovi, za odborné vedení, konzultace, trpělivost a podnětné návrhy k práci.

Obsah

Úvod	15
1 Základy internetové komunikace	16
1.1 OSI model	16
1.2 TCP/IP model	17
2 Útoky na odepření služeb	19
2.1 Distribuované útoky na odepření služeb	19
2.2 Záplavové DoS	20
2.2.1 ICMP Flood	21
2.2.2 UDP Flood	21
2.2.3 SYN Flood	23
2.2.4 HTTP Flood	24
2.2.5 ACK Flood	24
2.3 Logické DoS	24
2.3.1 Slowloris	25
2.3.2 Slow READ	26
2.3.3 Slow POST (RUDY)	26
2.3.4 Slow DROP	27
2.3.5 Slow NEXT	28
3 Problematika detekce a testování	30
4 Technologie pro vytvoření aplikace	31
4.1 Použití jazyka Python pro monitoring serverů	31
4.2 Zpracování a zobrazení nasbíraných dat	32
4.3 Ukládání dat	32
5 Implementace technologií	34
5.1 Postup vývoje	34
5.2 Sběr dat pomocí Python skriptů	34
5.2.1 Monitoring serverových prostředků	35
5.2.2 Měření odezvy serveru	36
5.3 Zpracování dat a webová aplikace	37
5.4 Fungování aplikace jako celku	38
5.5 Představení virtualizovaného pracoviště	39
5.5.1 Virtuální stroj oběti	40

6	Určení metod detekce DoS	41
6.1	Detekce založená na signaturách	41
6.2	Detekce založená na anomáliích	41
6.3	Falešně pozitivní výsledky detekce	42
6.4	Detekce pomocí vlastní aplikace	43
6.5	Metoda sledování TCP spojení	61
6.5.1	Testování metody	64
6.6	Metoda sledování poměru paketů a množství dat	65
6.6.1	Testování metody	67
6.7	Závislost mezi hardwarovými a softwarovými zdroji	68
6.7.1	Závislost při záplavovém útoku	69
6.7.2	Závislosti při logickém útoku	69
	Závěr	71
	Literatura	72
	Seznam symbolů a zkratk	75
	Seznam příloh	76
A	Obsah elektronické přílohy	77
B	Návod k instalaci vytvořené aplikace	78
B.1	Příprava monitorovacích skriptů	78
B.2	Instalace webové aplikace SMT	78
B.2.1	Instalace Node.js	78
B.2.2	Instalace databáze	78
B.3	Spuštění aplikace	79
B.3.1	Ověření funkčnosti	79
B.4	Propojení scriptů s aplikací	79

Seznam obrázků

1.1	Model OSI	16
1.2	Topologie modelu TCP/IP.	17
1.3	Protokoly vrstev TCP/IP.	18
2.1	DDoS útok za pomoci botnetu	20
2.2	Záplavové útoky, při kterých má útočník větší výpočetní výkon.	20
2.3	Princip útoku ICMP flood.	21
2.4	Rozdíl mezi TCP a UDP	22
2.5	Princip útoku UDP flood.	23
2.6	Princip navázání spojení v TCP protokolu.	23
2.7	Princip útoku SYN flood za pomoci polootevřených spojení.	24
2.8	Rozdíl mezi běžným provozem a slowloris útokem.	25
2.9	Princip útoku Slow READ	26
2.10	Princip útoku Slow POST	27
2.11	Princip útoku Slow DROP	28
2.12	Princip útoku Slow Next	29
5.1	Schema fungování aplikace	37
5.2	Úvodní stránka webové aplikace	38
5.3	Virtualizace strojů pomocí Proxmox	39
6.1	Změna zatížení jádra v průběhu ICMP Flood útoku	44
6.2	Nárůst datového toku internetového rozhraní v průběhu ICMP Flood útoku	45
6.3	Aktivní TCP spojení při ICMP Flood útoku	45
6.4	Nárůst zatížení procesoru důsledkem útoku UDP Flood	46
6.5	Zvýšení doby odezvy při útoku UDP Flood	46
6.6	Stavy TCP socketů při útoku SYN Flood	47
6.7	Množství přijatých a odeslaných paketů při testování SYN Flood	48
6.8	Výrazné zatížení CPU při útoku HTTP Flood	49
6.9	Graf TCP stavů na portu 80 při útoku HTTP Flood	49
6.10	Zatížení síťového rozhraní při testování dopadů HTTP Flood	50
6.11	Souměrné zatížení sítě v obou směrech v důsledku ACK Flood	51
6.12	Stav TCP spojení v průběhu ACK Flood útoku	52
6.13	Stavy TCP spojení při útoku Slowloris	52
6.14	Změřená odezva serveru v průběhu útoku Slowloris	53
6.15	Stavy TCP spojení v průběhu útoku Slow READ	54
6.16	Množství procházejících dat síťovým rozhraním oběti při útoku Slow READ	55
6.17	Apache scoreboard při útoku Slow READ	55

6.18	Zátěž CPU během útoku Slow POST	56
6.19	Jednotlivé stavy TCP spojení při útoku Slow POST	57
6.20	Zatížení síťového rozhraní při útoku Slow DROP	58
6.21	Počet paketů za sekundu během útoku Slow DROP	58
6.22	Nárůst zatížení procesoru při útoku Slow NEXT	59
6.23	Vliv Slow NEXT útoku na aktivní TCP spojení	60
6.24	Apache scoreboard při útoku Slow NEXT	60
6.25	Zobrazení závislosti počtu aktivních spojení a jejich změně na čase od začátku testu útoku SYN flood	62
6.26	Lineární nárůst spojení TCP při testu útoku Slow READ	62
6.27	Zobrazení konstantní velikosti paketů při ICMP Flood útoku	65
6.28	Útok Slow READ, který není metodou sledování velikosti paketů de- tekován	67
6.29	Zobrazení závislosti počtu spojení na vytížení RAM a CPU v průběhu SYN Flood	69
6.30	Zobrazení závislosti počtu spojení na vytížení RAM a CPU v průběhu Slowloris útoku	70

Seznam tabulek

5.1	Popis konfiguračních parametrů souboru <code>server_script.ini</code>	35
5.2	Popis konfiguračních parametrů souboru <code>network_script.ini</code>	36
5.3	Přehled IP adres dle rozhraní.	40
6.1	Testování metody sledováním TCP spojení	64
6.2	Testování metody velikosti paketu	68

Úvod

Tato bakalářská práce se věnuje metodám sloužícím k popisu stavu odepření služby DoS (z anglického Denial of Service) a jejich následné implementaci. Identifikace útoků typu DoS je v současnosti velmi aktuálním tématem, neboť se tyto útoky staly běžnou součástí kyberprostoru a jejich počet neustále roste [1]. V prvním čtvrtletí roku 2023, souběžně s válkou na Ukrajině, došlo k výraznému nárůstu distribuovaných útoků typu DoS na média, státní úřady, telekomunikační společnosti a další organizace. Podle portálu Živě.cz byl v tomto období zaznamenán útok trvající 177 hodin, což ilustruje závažnost a vytrvalost těchto útoků [9]. Efektivní identifikace podobných útoků je klíčová pro ochranu před nimi a pro nalezení včasného záložního řešení pro kritické služby. Tato práce se zaměřuje na útoky v rámci lokálních sítí a na webový Apache server, kde je možné detailně analyzovat různé typy DoS útoků a jejich dopad.

Nejprve se práce zajímá o úvod do síťové komunikace, zahrnující protokoly TCP/IP a model ISO/OSI. Tento základ poskytuje čtenářům potřebné porozumění o fungování síťových protokolů, což je klíčové pro pochopení následných kapitol o útocích DoS.

Následuje teoretické seznámení s různými typy útoků DoS, včetně popisu botnetů a jednotlivých typů útoků. Tato část práce rozebírá principy, na kterých jsou tyto útoky založeny, a vysvětluje, jak různé techniky mohou být využity k přetížení cílových systémů a služeb.

Další část představuje technologie použité při implementaci řešení a přechází v popis implementace a testovacího virtualizovaného prostředí. V této části je detailně popsán postup vývoje, konfigurace testovacího prostředí a použití různých nástrojů a technologií. Virtualizované prostředí umožňuje bezpečné testování různých typů útoků a analýzu jejich dopadu na serverové zdroje.

V závěrečné části práce se zaměřuje na metody detekce DoS útoků a vysvětluje, jak může vyvinutá aplikace pomoci při detekci těchto útoků. Následně je ukázáno, jak různé typy útoků působí na servery, a prezentovány výsledky testování a analýzy. Závěrem je zahrnuta úvaha nad souvislostí zatížení hardwarových a softwarových prostředků a jejich vlivem na identifikaci útoků.

Celkově tato práce poskytuje komplexní přehled o problematice DoS útoků, od teoretických základů přes detekční metody až po praktické testování a vizualizaci dat. Cílem je poskytnout čtenáři hlubší porozumění těmto útokům a nástroj pro jejich efektivní detekci.

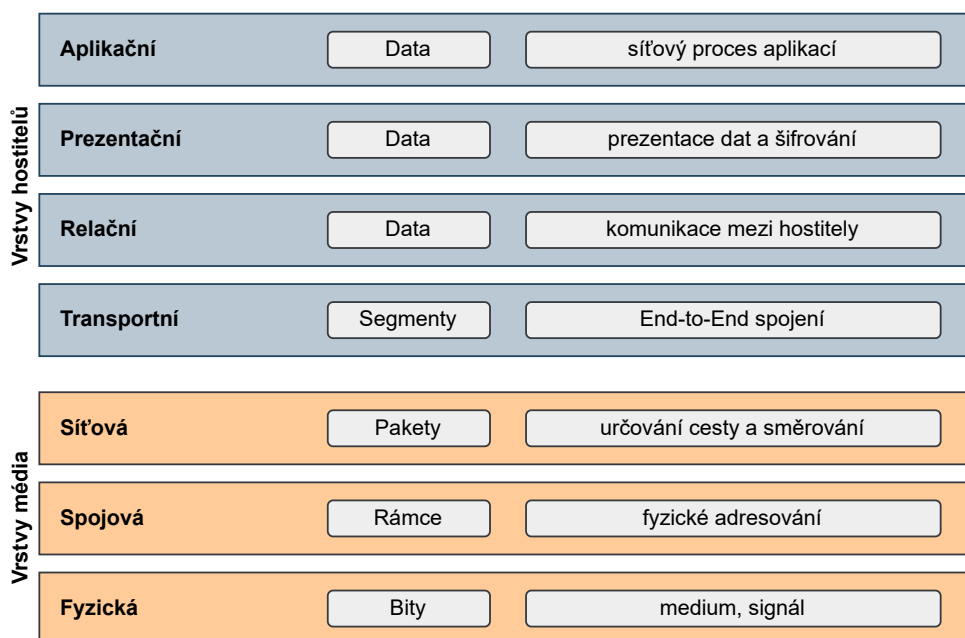
1 Základy internetové komunikace

V éře digitální transformace se Internet stává stále více nezbytným pro naši každodenní komunikaci, obchodování, volnou zábavu i vzdělávání. V minulosti byla data přenášena pomocí analogových signálů přes telefonní síť. S nástupem digitálního věku se ale přenos přeorientoval na digitální signály, což umožnilo rychlejší, bezpečnější a efektivnější komunikaci.

Základem dnešní síťové komunikace jsou modely, které definují pravidla (protokoly) pro výměnu dat. Nejznámějšími modely jsou *Open Systems Interconnection* (OSI) a *Transmission Control Protocol/Internet Protocol* (TCP/IP). Zatímco OSI model poskytuje teoretický rámec pro komunikaci mezi různými síťovými systémy, TCP/IP je praktickým modelem, který Internet využívá. [3]

1.1 OSI model

OSI model představuje teoretický rámec pro vzájemnou komunikaci různých síťových systémů. Skládá se ze sedmi vrstev, od fyzické až po aplikační, přičemž každá vrstva specifikuje určité aspekty síťových operací a poskytuje služby vrstvě nad ní. Celkově se nám tak nabízí komplexní přehled o tom, jak by měla síťová komunikace probíhat.

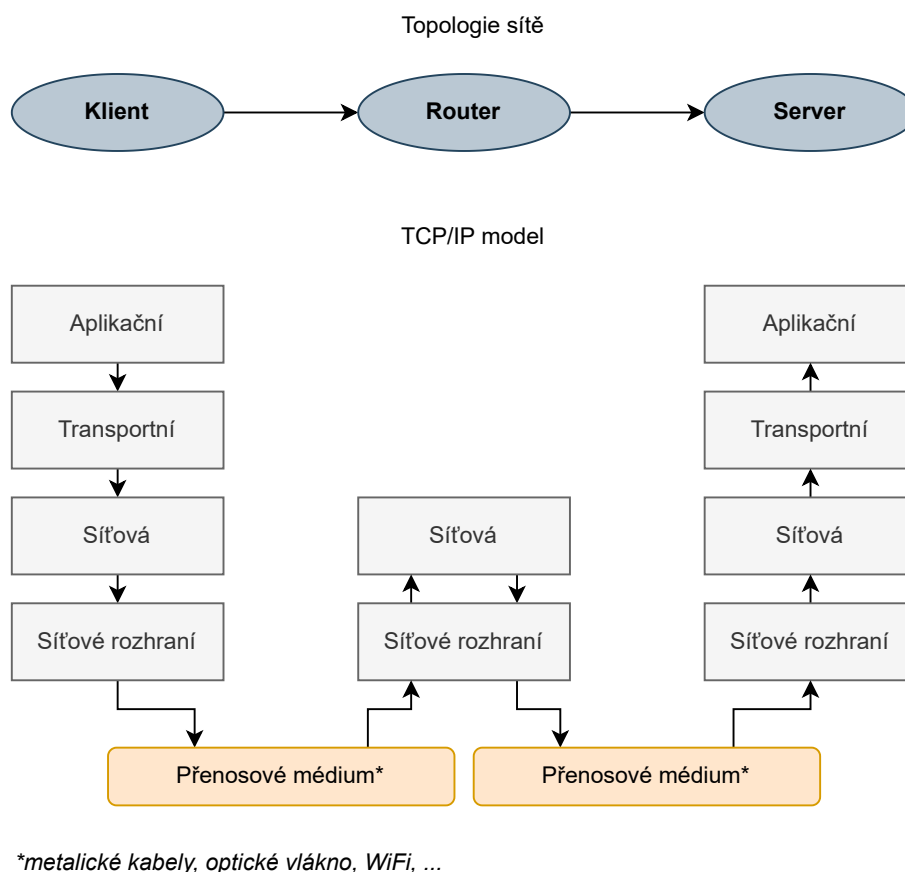


Obr. 1.1: Model OSI

Tento model slouží jako univerzální vzdělávací nástroj a referenční model pro pochopení a návrh sítí. Jeho vrstvená architektura umožňuje vývojářům a síťovým inženýrům izolovat a řešit problémy specifické pro jednotlivé vrstvy. Díky tomu je možné lépe diagnostikovat a opravovat problémy související se sítěmi a zároveň zjednodušovat návrh a implementaci nových technologií a protokolů. [3]

1.2 TCP/IP model

Na druhé straně, TCP/IP je praktický model (obr. 1.2), který je přímo využíván Internetem. Byl vyvinut v 70. letech 20. století americkou obrannou agenturou DARPA a stál u zrodu Internetu. Samotný model se skládá ze čtyř vrstev: linkové vrstvy, internetové vrstvy, transportní vrstvy a aplikační vrstvy. [18]

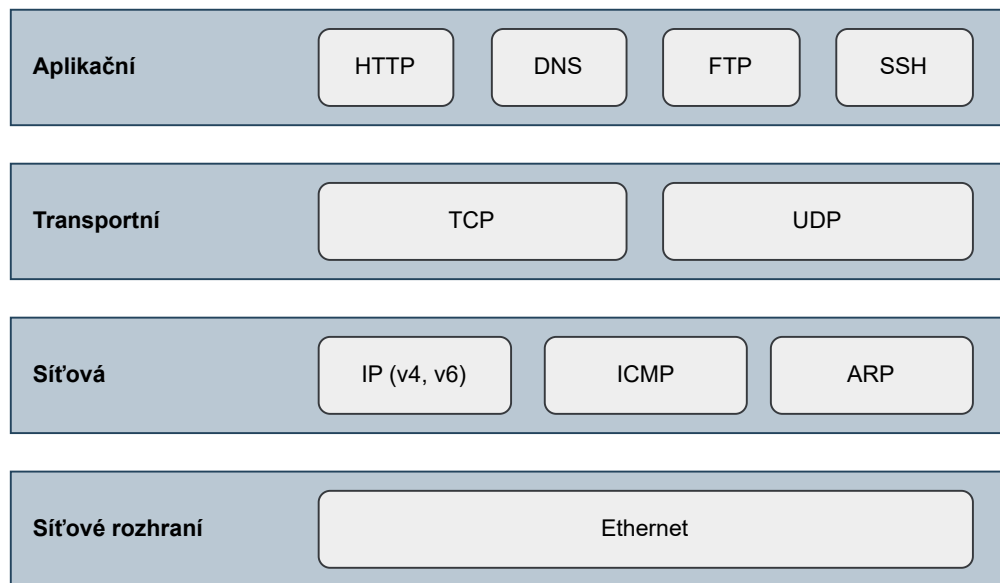


Obr. 1.2: Topologie modelu TCP/IP.

Internetová vrstva, kde dominuje protokol *Internet Protocol* (IP), se stará o adresování a směrování dat mezi zařízeními. Transportní vrstva pak zahrnuje protokoly *Transmission Control Protocol* (TCP) a *User Datagram Protocol* (UDP), kde TCP zajišťuje spolehlivý přenos dat, zatímco UDP nabízí rychlejší, ale méně spolehlivé

spojení. Aplikační vrstva pak umožňuje uživatelům interakci s konkrétními aplikacemi na Internetu, jako jsou webové stránky, e-mailové služby nebo přenos dat přes protokoly jako je *Hypertext Transfer Protocol* (HTTP), *File Transfer Protocol* (FTP) nebo *Secure Shell* (SSH).

Nicméně, jednotlivé protokoly používané na prvních třech vrstvách modelu TCP/IP vykazují určité slabiny v jejich implementaci, které mohou být zneužity při *Denial of Service* (DoS) útocích. Tyto útoky specificky cílí na vyčerpání systémových nebo síťových zdrojů právě využitím zranitelností těchto protokolů. [21]



Obr. 1.3: Protokoly vrstev TCP/IP.

2 Útoky na odepření služeb

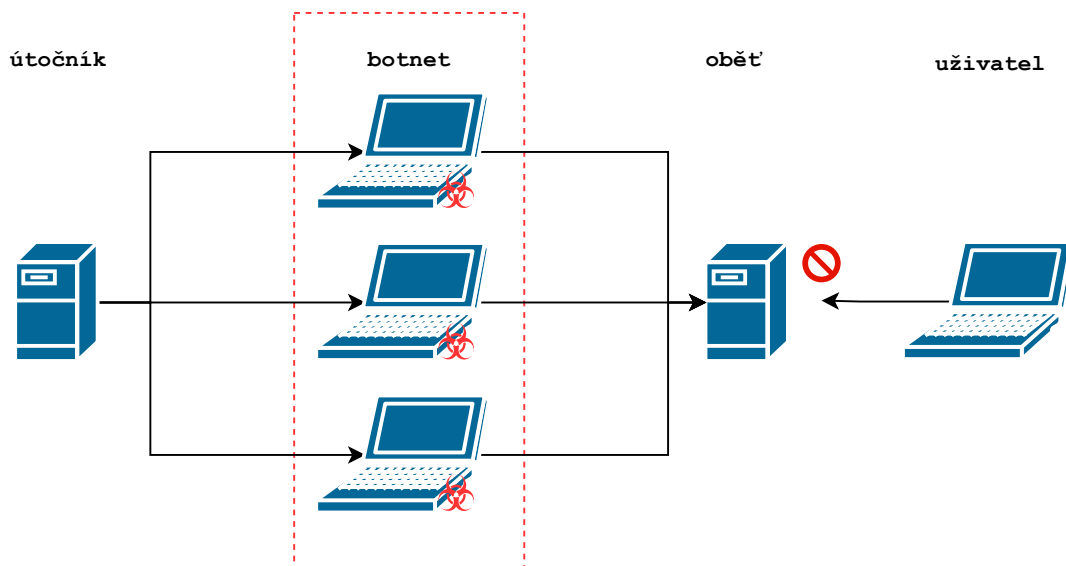
Jedná se o velmi jednoduchý útok, jehož cílem je zamezit, případně alespoň zpomalit rychlost přístupu legitimního uživatele k serveru, webové aplikaci nebo síti. Útok se snaží využít veškeré dostupné hardwarové a softwarové prostředky těchto systémů a dosáhnout jejich přetížení. Tím, že nezbude žádná volná kapacita, není možné případného uživatele obsloužit a dojde tak k odepření služby. Pro tyto útoky se používá označení DoS.

Obětí těchto útoků se může stát cokoliv, ať už je to internetový obchod, webový portál banky, stránky státních institucí nebo zpravodajské služby. Tento útok ale není nebezpečný ve smyslu ukradení dat, získání kontroly nad koncovým zařízením či neautorizovaným přístupem. Jeho nebezpečí spočívá ve ztrátě uživatelů aplikace, potenciálních kupujících či nemožnosti navštívení webů s aktuálním zpravodajstvím. Dobře nachystaný DoS v kombinaci se slabší ochranou služby dokáže například odstavit mimo provoz platební bránu banky a ač by nedošlo k žádnému ukradení prostředků i tak budou škody nevyčísitelné. Pro útočníky tak může být motivací oslabení konkurence, vyjednání finanční odměny za ukončení útoku nebo politické přesvědčení. Není to tak dávno, co v pondělí 10. října 2022 napadla ruská hackerská skupina Killnet DoS útokem služby amerických letišť a například internetové rezervace parkování byly pro uživatele nedostupné [14].

2.1 Distribuované útoky na odepření služeb

Distributed Denial of Service (DDoS), neboli distribuovaný DoS, je pokročilejší variantu útoku na odepření služeb. Místo jednoho počítače zahlcuje server požadavky několik dalších počítačů. Je zde dokonce možnost, že se i běžný počítač může stát součástí takovýchto sítí a to dokonce tak, že si toho nemusí uživatel ani všimnout. Takto napadené počítače se nazývají „zombies“ a dohromady tvoří „botnet“¹. Náhorný příklad takovýchto útoků je vidět na obrázku 2.1. V této práci se ale zaměříme pouze na metody detekce nedistribuovaných, tedy standardních DoS útoků. [23]

¹Síť zařízení připojených k internetu, které jednají automaticky nebo autonomně.

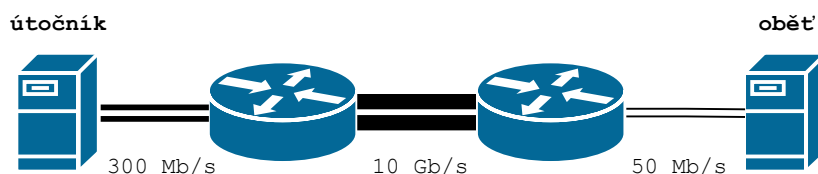


Obr. 2.1: DDoS útok za pomoci botnetu

2.2 Záplavové DoS

Jedná se nejobyčejnější možné útoky, které předpokládají výpočetní převahu útočníka nad obětí. Jejich cílem je generovat velké množství požadavků a vytvářet tak velmi vysoký datový tok, který přehltí síť nebo server oběti a ta tak není schopna požadavky zpracovat. Na obrázku 2.2 můžeme vidět příklad DoS útoku, který spoléhá na nižší přenosovou rychlost připojení na straně oběti.

Velkou nevýhodou těchto útoků je jejich poměrně jednoduchá identifikovatelnost a to prostřednictvím sledování zátěže hardwarových zdrojů serveru. Z důvodu malé efektivity tohoto útoku a taky kvůli faktu, že jsou velké servery připojeny na dostatečně rychlých připojení je pravděpodobnost úspěchu záplavového DoS velmi malá. V dnešní době je ale již tento způsob útoků překonán.



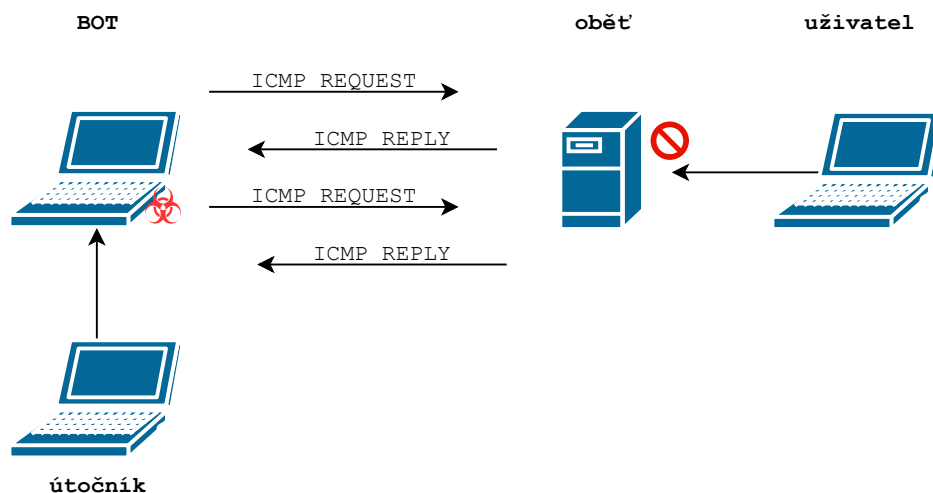
Obr. 2.2: Záplavové útoky, při kterých má útočník větší výpočetní výkon.

2.2.1 ICMP Flood

Během tohoto útoku se útočník snaží zahltit cílový server prostřednictvím paketů protokolu *Internet Control Message Protocol* (ICMP). Jedná se o protokol síťové vrstvy, který se používá pro hlášení chyb spojení. Například při velké velikosti paketu může dojít v routeru k jeho zahození a router pak odešle ICMP zprávu zpět zdroji původního paketu. Dále se s tímto protokolem setkáme v diagnostických konzolových aplikacích, konkrétně při příkazu `ping` nebo `tracert`. [27]

První zmíněný příkaz je právě použit v tomto útoku, proto se také někdy tento útok nazývá Ping Flood. Tento požadavek vyžaduje na straně serveru jak určité prostředky tak i šířku pásma pro příchozí a odchozí zprávu. Cílem útoku je dostat server do stavu, kdy nebude schopen na vysoký počet požadavků reagovat a vyčerpá tak své zdroje, přetížit síťové připojení serveru tímto generovaným falešným provozem. [8]

Dříve se běžně u tohoto útoku používala podvržená zdrojová adresa s cílem zachování anonymity a maskování útočníka. Dnes se spíše ale setkáváme s variantou kdy na server hacker útočí prostřednictvím botů. Na obrázku 2.3 můžeme vidět demonstrativní případ takového útoku.

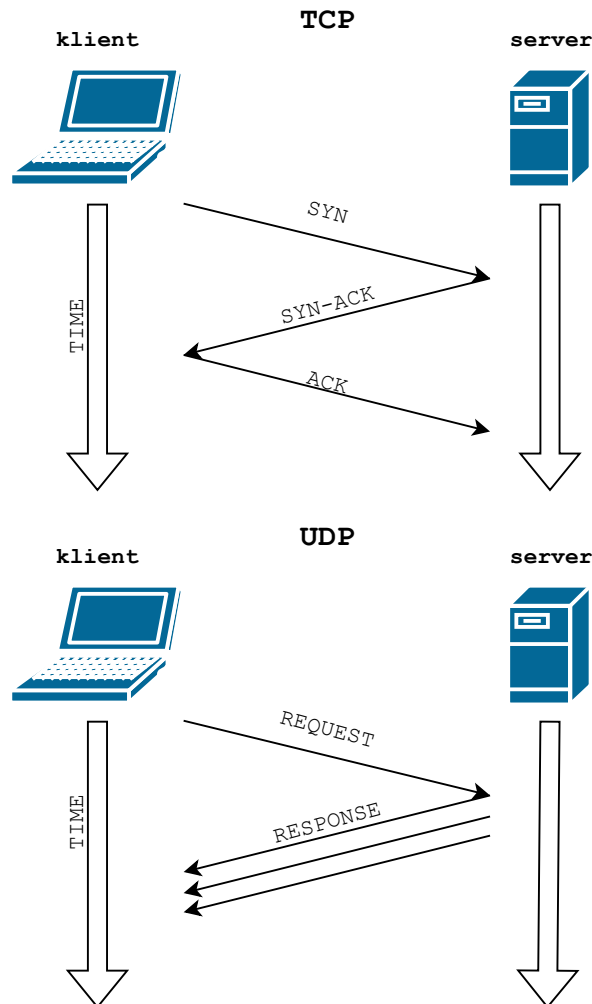


Obr. 2.3: Princip útoku ICMP flood.

2.2.2 UDP Flood

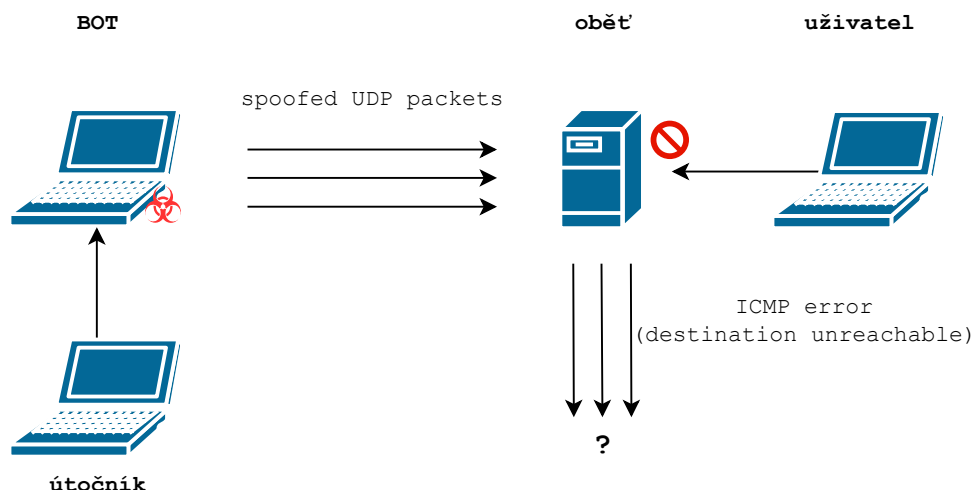
Je dalším ze série záplavových útoků. Stejně jak předchozí útoky i tento cílí krom samotného serveru na celou síť před ním. Pro svoji činnost využívá UDP protokol, který se používá pro přenosy dat, které jsou potřeba přepřakovat s co nejmenším

zpožděním. Oproti TCP tak zrychluje komunikaci tím, že nesestavuje spojení a přenos paketů není ověřován. Díky těmto vlastnostem si našel místo například u streamingu videa, nebo přenosu stavových informací. Rozdílný princip těchto protokolů je znázorněn na obrázku 2.4.



Obr. 2.4: Rozdíl mezi TCP a UDP

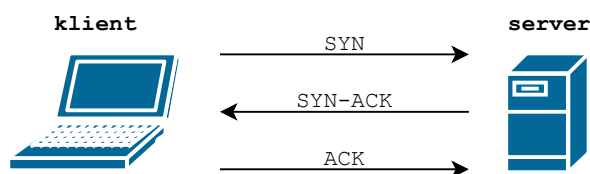
Samotný útok primárně využívá kroky, které server učiní po přijetí UDP paketu. Za běžných podmínek server zkontroluje, zda na daném portu nenaslouchá nějaká aplikace, která je ochotna přijímat pakety, aby jí mohl paket předat. V opačném případě pak server odpoví pomocí paketu ICMP Destination Unreachable s informací že cíl byl nedostupný. Lze tedy říci že server zde plní úlohu prostředníka, který směřuje pakety ke správným aplikacím, přičemž vždy musí ověřit zadané podmínky. Již zmíněné kroky, které server musí učinit při přijetí každého jednoho paketu, se projeví na využití prostředků daného serveru. Na obrázku 2.5 můžeme vidět příklad, kdy útočník nahradí zdrojovou IP adresu a pro útok využije bota. [20]



Obr. 2.5: Princip útoku UDP flood.

2.2.3 SYN Flood

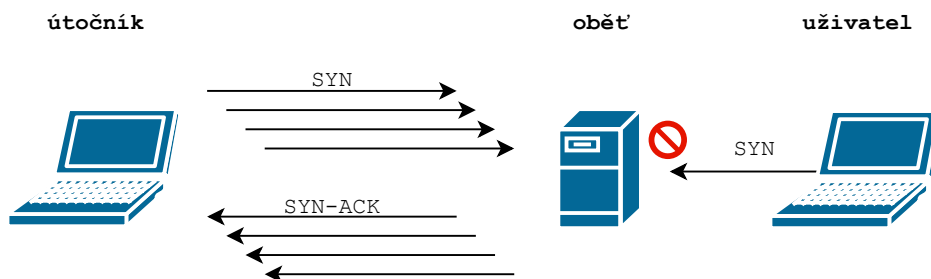
Útok SYN flood využívá vlastností protokolu TCP a to konkrétně navazováním jeho spojení. To probíhá za použití techniky „3-way handshake“, při které klient iniciuje spojení. Průběh tohoto procesu je znázorněn na obrázku 2.6. Při běžném



Obr. 2.6: Princip navázání spojení v TCP protokolu.

provozu je na klientovu výzvu SYN odpovězeno SYN-ACK a po odpovědi klienta ACK je ustanoveno spojení mezi oběma stranami. Informace o vytvořeném spojení se ukládají prostřednictvím *Transmission Control Block* (TCB).

Právě této skutečnosti se při útoku využívá. Útočník inicializuje velké množství SYN požadavků, na které mu server následně odešle odpovědi, uloží polootevřené spojení a bude vyčkávat na ACK. Tohoto potvrzení se ale již nedočká. Serverová odpověď SYN-ACK útočnickovy vůbec nedorazí, protože je podvržená zdrojová IP adresa, nebo ji útočník zcela ignoruje. Polootevřená spojení uložená na serveru zabírají postupně celé místo, které má server k dispozici, z toho důvodu pak dojde k odepření služby legitimnímu uživateli, jelikož na uložení jeho spojení již nezbude místo. Tento průběh je znázorněn na obrázku 2.7. [19]



Obr. 2.7: Princip útoku SYN flood za pomoci polootevřených spojení.

2.2.4 HTTP Flood

Tento útok je další z řady volumetrických útoků vytvořený pro zahlcení serveru HTTP požadavky. Útok probíhá na aplikační vrstvě a je cílen na webový server. Tento útok je obzvláště nebezpečný, protože využívá standardní HTTP požadavky, které nelze u webového serveru blokovat. V praxi je tento útok většinou provozován distribuovaně. [5]

2.2.5 ACK Flood

Tento útok využívá principů TCP, konkrétně třístupňového procesu navazování spojení známého jako TCP handshake. V normálním provozu TCP, když klient komunikuje se serverem, začne tím, že odešle SYN (synchronize) paket k iniciaci spojení. Server odpoví SYN-ACK (synchronize-acknowledge) paketem, a klient dokončí proces tím, že pošle ACK paket. Jakmile je toto spojení navázáno, mohou mezi klientem a serverem bezpečně probíhat data.

Při ACK flood útoku útočník zasílá masivní množství nevyžádaných ACK paketů bez předchozího navázání skutečného spojení. Tyto pakety mají často nesprávné, padělané nebo náhodné sekvenční čísla, což nutí cílový server zpracovávat a reagovat na každý z nich. Server musí na každý paket reagovat a pokusit se zjistit, zda patří k existujícímu spojení, což vyžaduje značné množství výpočetních a paměťových zdrojů. [24]

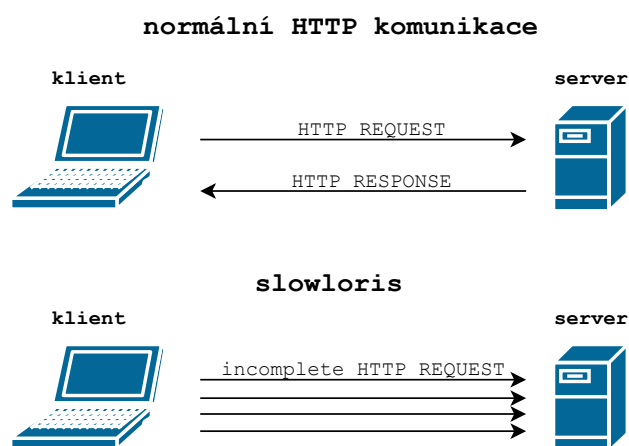
2.3 Logické DoS

Logické útoky na dostupnost služeb se vyznačují specifickým využitím charakteristik a chování cílového systému, protokolu nebo aplikace s cílem narušit jejich funkčnost. Na rozdíl od záplavových útoků, které jsou založeny na zahlcení sítě velkým objemem dat, logické útoky vyžadují relativně menší množství dat a též dosahují stavu odepření služby. Efektivita těchto útoků je založena na zneužití logických chyb nebo

zranitelností, které se nacházejí v cílovém systému. Tyto útoky jsou obvykle sofistikované a obtížně detekovatelné, jelikož se podobají běžnému provozu a chování legitimního uživatele. [26]

2.3.1 Slowloris

Jedná se o program a po něm pojmenovaný útok, který probíhá na aplikační vrstvě TCP/IP modelu. Útočník se snaží přemoci server otevřením velkého množství HTTP² spojení a jejich následnému udržování. Pro jeho funkci se využívají dílčí požadavky protokolu. Činnost programu spočívá v postupném otevírání připojení k cílovému webovému serveru a jejich obnovování prostřednictvím nekompletních HTTP requestů tak, aby nedošlo k jejich ukončení kvůli překročení časového limitu připojení. [15] Ve výsledku se tak tváří jako velmi pomalý klient, který je ale stále připojen. Porovnání se standardním provozem HTTP je k dispozici na obrázku 2.8.



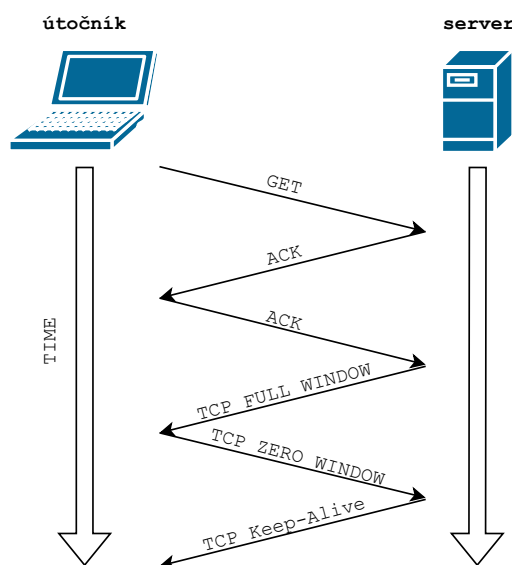
Obr. 2.8: Rozdíl mezi běžným provozem a slowloris útokem.

Slowloris nelze brát jako nějakou kategorii útoku, ale přímo program, navržený tak, aby zahltil server požadavky, způsobem který nespotřebuje velké množství šířky pásma, kterou má server k dispozici. Na rozdíl od předchozích útoků, je tento více podobný běžnému síťovému provozu a je tedy hůře identifikovatelný.

²Hypertext Transfer Protocol je základem World Wide Webu a používá se k načítání webových stránek pomocí hypertextových odkazů. HTTP je protokol aplikační vrstvy určený k přenosu informací mezi síťovými zařízeními.

2.3.2 Slow READ

Útok Slow READ využívá HTTP požadavky typu GET, kterými z webového serveru stahuje obrázky či soubory téměř nulovou rychlostí. Cílem je udržet spojení mezi klientem a serverem co nejdéle aktivní, což vede k zatěžování serverových zdrojů. Tímto způsobem se může zvýšit počet současně otevřených spojení, což může vést k vyčerpání dostupných zdrojů serveru a způsobit jeho zpomalení nebo dokonce nedostupnost pro legitimní uživatele.



Obr. 2.9: Princip útoku Slow READ

Velkou výhodou útoku Slow READ pro útočníka je, že během tohoto útoku je provoz na síti mezi útočníkem a serverem minimální. Díky tomu je složitější tento útok detekovat pomocí tradičních metod, které sledují vysoký síťový provoz jako indikátor útoku. Útočník může tímto způsobem využít malé množství síťových prostředků k dosažení relativně významného dopadu na cílový server.[17]

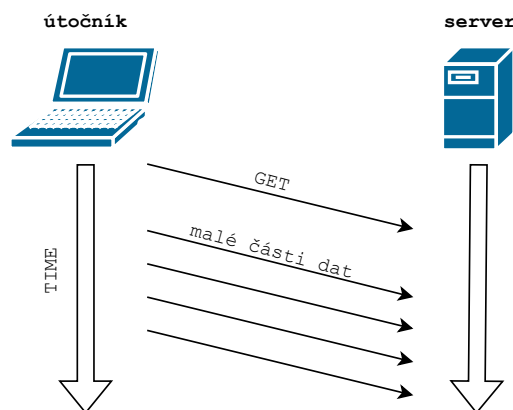
Pro detekci Slow READ útoků mohou být použity různé metody, například monitorování neobvykle dlouhých HTTP spojení, limit pro čtení dat a nebo hlídání minimální přenosové rychlosti.

2.3.3 Slow POST (RUDY)

Tento útok, známý také pod názvem RUDY (R-U-Dead-Yet), využívá metodu POST, kterou nabízí protokol HTTP. Cílový webový server je zahlcen legitimními hlavič-

kami protokolu, ve kterých je definována velikost těla zprávy, které bude na server ještě odesláno. Útočník ale odesílá data velmi nízkou rychlostí. Podle webu NET-SCOUT klidně i 1 b každé 2 minuty [16]. Dnes ale musí být pro úspěšný útok tato frekvence vyšší, protože poslední verze serverů již mají upravenou konfiguraci a nečekají tak dlouho. Zároveň se pro snížení viditelnosti útoku tento čas pro každý požadavek náhodně generuje. Ve výsledku tak vzniká provoz 1 B za každých přibližně 10 vteřin [13].

Tyto zprávy jsou zpracovávány zcela standardně, jelikož se může pouze jednat o pomalé připojení na straně klienta. Podobně jako u předchozích útoků dochází nejprve ke zpomalení serveru, a pokud útočník naváže spojení ve stovkách nebo dokonce tisících, může se stát server pro legitimního uživatele nedostupným.

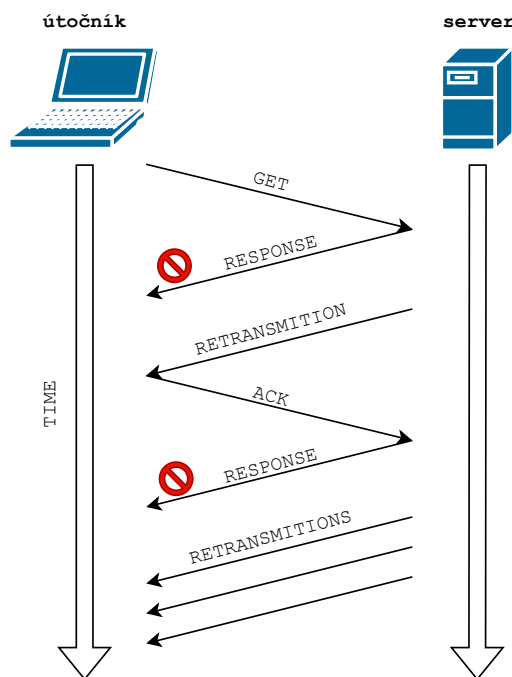


Obr. 2.10: Princip útoku Slow POST

2.3.4 Slow DROP

Útok Slow DROP je zaměřen na využití vlastností protokolu TCP k vyčerpání serverových zdrojů. Útočník v tomto případě přijímá pakety od serveru a některé z nich náhodně zahazuje. Tímto způsobem se server stává nuceným opakovaně odesílat komunikaci, což výrazně zvyšuje jeho zátěž.

Samotný protokol TCP je navržen tak, aby zajišťoval spolehlivé doručení dat mezi dvěma koncovými body. Při každém odeslání paketu server očekává potvrzení o jeho přijetí. Pokud k potvrzení nedojde v daném časovém intervalu, server předpokládá, že paket byl ztracen a data opětovně odesílá. Tímto způsobem je vytvořen velký provoz na straně serveru s minimální účastí útočníka. [2]

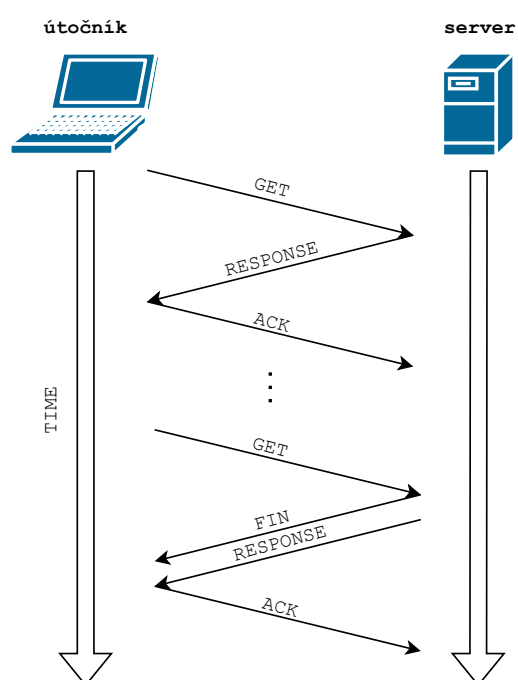


Obr. 2.11: Princip útoku Slow DROP

2.3.5 Slow NEXT

Jedná se o útok, který se zaměřuje na zpomalování zpracování požadavků serverem a udržování dlouhých otevřených spojení. Útočník při tomto útoku naváže legitimní TCP spojení s cílovým serverem a poté čeká na odpověď. Jakmile server zpracuje požadavek a odešle odpověď, spojení přejde do stavu `FIN_WAIT1`, kde server čeká na potvrzení od klienta. Útočník však neukončí spojení okamžitě, ale místo toho před vypršením timeoutu odešle platný udržovací požadavek, čímž udržuje spojení aktivní po delší dobu.

Cílem tohoto útoku je vyčerpat serverové zdroje zahlcením serveru velkým množstvím požadavků, na které musí postupně reagovat, ale spíš jejich udržením je v aktivním stavu. Tím, že útočník pravidelně odesílá udržovací požadavky těsně před vypršením časového limitu, který je definován parametrem `KeepAliveTimeout`, zabráňuje serveru uzavřít spojení a nutí jej alokovat a spravovat prostředky pro každé z těchto spojení. Tento postup způsobuje zdržení zpracování nových požadavků a vede tak ke stavu odepření služeb. [2]



Obr. 2.12: Princip útoku Slow Next

3 Problematika detekce a testování

Detekce DoS útoků představuje jednu z klíčových oblastí v oblasti kybernetické bezpečnosti, která vyžaduje neustálý výzkum a rozvoj kvůli častým změnám v technologiích a útočných metodách. Samotné testování efektivity detekčních systémů je zásadní pro zajištění jejich spolehlivosti a efektivity. Testování by mělo pokrývat různé scénáře, které mohou nastat při skutečných útocích, včetně různých typů DoS útoků jako jsou záplavové útoky, útoky na protokoly a útoky na aplikaci. K tomuto účelu se využívají nástroje a simulace, které generují vysoký objem provozu nebo požadavků s cílem ověřit, jak daný systém reaguje.

Stanovení pravidel pro detekci DoS je nutné pro zajištění, že bezpečnostní opatření jsou účinná a minimalizují falešně pozitivní výsledky. Pravidla by měla být založena na analýze historických dat, znalostech o běžných útočných vektorech a aktuálních trendech v kybernetických útocích. Dále je důležité tyto pravidla pravidelně aktualizovat, aby odpovídala nejnovějším hrozbám a změnám v síťovém prostředí. Pravidla by měla být nastavena tak, aby efektivně oddělovala běžný provoz od potenciálně škodlivého, přičemž by měla zahrnovat parametry jako jsou frekvence požadavků, velikost paketů, chování zdrojových IP adres a typy protokolů.

Abychom mohli útoky efektivně detekovat musíme vědět jak probíhají a jak se na napadeném serveru projevují. Právě za tímto účelem byla vytvořena aplikace, která je součástí této práce a díky níž máme real-time monitoring zatížení hardwarových a softwarových zdrojů. Měření rychlosti odezvy serveru ze sítě může sloužit jako ukazatel toho, že došlo ke stavu odepření služby. Všechny tyto údaje v grafech umožňují rychlou identifikaci neobvyklých vzorců, které by mohly naznačovat probíhající útok.

4 Technologie pro vytvoření aplikace

V rámci této bakalářské práce byla vytvořena aplikace, která poskytuje uživateli přehled o využití softwarových a hardwarových prostředků. Tato aplikace se skládá ze dvou skriptů pro sběr dat, webové aplikace sloužící jako grafické uživatelské rozhraní a databáze pro uchovávání historických dat měření.

V následujících podkapitolách budou podrobně představeny jednotlivé technologie použité při vývoji této aplikace, jejich historie a možnosti, které nabízejí. Implementace těchto technologií a jejich integrace do jednoho celku budou popsány v kapitole 5.

4.1 Použití jazyka Python pro monitoring serverů

Python je v současné době jedním z nejpoužívanějších programovacích jazyků, známý svou univerzálností a širokými možnostmi použití v různých oblastech, včetně monitoringu serverů. Jako moderní, vysokoúrovňový jazyk podporuje Python různé programovací paradigma - procedurální, funkcionální a objektově orientované programování. Jeho historie sahá do roku 1991, kdy byl poprvé představen Guidem van Rossumem a od té doby se stal zásadním nástrojem pro vývoj softwaru díky své jednoduchosti a čitelnosti. [12]

Původně byl tento jazyk navržený pro výuku programování a prototypování aplikací rychleji než v jazyce C. Python pak získal popularitu v technických a výzkumných komunitách, což vedlo k rozvoji široké škály knihoven určených pro specifické účely. V oblasti monitoringu serveru poskytuje Python bohatou sadu knihoven jako například `psutil`, která slouží pro získávání informací o systémových procesech a výkonu.

Pro efektivní monitoring je nezbytné sbírat data v reálném čase, což vyžaduje schopnost paralelního zpracování. Python nabízí na tyto potřeby knihovny jako `threading` a `concurrent.futures`, které umožňují multivláknové zpracování. Tyto knihovny poskytují základ pro souběžné sbírání dat z různých zdrojů bez významného zpomalení hlavního provozu aplikace. Pomocí více vláken je totiž možné vytvořit periodické úlohy, které budou pravidelně měřit a sbírat data různých systémových zdrojů jako CPU, RAM, síťového provozu nebo I/O operací. Tato data mohou být následně analyzována a využita pro detekci anomálií, monitorování výkonu systému nebo vytváření statistik.

4.2 Zpracování a zobrazení nasbíraných dat

Webová prezentace nasbíraných hodnot bude v tomto projektu vytvořena pomocí Node.js. Jedná se o open-source, multiplatformní běhové prostředí pro JavaScript, které umožňuje spouštění kódu na serverové straně. Tento nástroj byl představen v roce 2009 Ryanem Dahlem a rychle získal popularitu mezi vývojáři pro jeho schopnost zvládat asynchronní I/O operace pomocí non-blocking¹, event-driven architektury². Node.js umožňuje vývojářům využívat JavaScript, tradičně jazyk pro frontend, i na backendu, čímž umožňuje vytváření univerzálních aplikací s jednotným programovacím jazykem. [11]

Node.js aplikace mohou snadno komunikovat s databázemi, což je zásadní pro aplikace, které přijímají data z externích zdrojů, jako jsou v tomto případě skripty Pythonu. Data jsou přijímána prostřednictvím API, zpracována a ukládána do databáze pro další použití. Tento proces je zjednodušen díky množství dostupných knihoven pro různé typy databází, jako jsou MongoDB, PostgreSQL nebo MySQL.

Pro vizualizaci dat ve webové aplikaci se často využívá knihovna `Chart.js`, která umožňuje vytváření interaktivních, responzivních grafů a diagramů a bude v této práci použita. Jedná se o jednoduchou, ale velmi rozsáhlou knihovnu, která poskytuje bohaté možnosti pro prezentaci datových sad v grafické formě. Právě zobrazení pomocí grafů je ideální pro dashboards a získáme tak jednoduše přehledné informace na rozdíl od tabulek.

Node.js se stal klíčovým hráčem v oblasti moderního webového vývoje. Jeho schopnost efektivně zpracovávat asynchronní operace, spolu s univerzálním použitím JavaScriptu pro obě části aplikace (front-end a back-end), činí Node.js ideální volbou pro vývoj komplexních, výkonných a snadno udržitelných webových aplikací.

4.3 Ukládání dat

Pro tento projekt byla vybrána databáze PostgreSQL, často označovaná jako Postgres. Jedná se o jeden z nejvyspělejších open-source objektově-relačních databázových systémů na světě. Díky své bohaté historii a pokračujícímu vývoji poskytuje

¹Tradiční synchronní I/O operace v jazyku jako je Java nebo C# blokují vykonávání dalšího kódu, dokud nejsou dokončeny. Node.js využívá non-blocking I/O operace, které umožňují pokračovat v běhu programu bez čekání na dokončení těchto operací. Pokud program potřebuje načíst soubor nebo získat data z databáze, pošle požadavek a pokračuje ve zpracování dalšího kódu. Když je operace dokončena, Node.js obdrží oznámení a zpracuje data v callback funkci.

²Node.js pracuje na bázi event loop (smyčka událostí), která umožňuje efektivní zpracování událostí. Event loop neustále kontroluje, zda jsou nějaké události k zpracování, a pokud ano, zpracuje je pomocí odpovídajících callback funkcí.

PostgreSQL vysokou úroveň kompatibility s normami SQL a je známý svou spolehlivostí, robustností a výkonem. Je ideální volbou pro velké aplikace vyžadující komplexní transakce a dotazy, a je široce využíván v průmyslu od malých start-upů až po velké korporace. [10]

5 Implementace technologií

V této části bakalářské práce se zaměříme na implementaci dříve zmíněných technologií. Nejprve představíme celkový postup vývoje a integrace potřebných technologických komponent. Projdeme jednotlivé fáze od sběru dat, přes jejich zpracování, až po vizualizaci výsledků. Následně se zaměříme na funkčnost aplikace a možnosti jejího praktického využití. Závěr této kapitoly je věnován popisu virtualizovaného pracoviště, na kterém byla aplikace testována a ověřena.

5.1 Postup vývoje

Úvodní část mé práce byla věnována vývoji skriptů, zaměřených primárně na sběr a filtrování dat. Postupně jsem testoval různé metody pro získávání klíčových metrik a jejich selektivní zobrazení, aby byla garantována relevace informací. Po úspěšné validaci metod sběru dat jsem přistoupil k integraci funkcí do jediného serverového skriptu, pro který jsem implementoval vícevláknové zpracování.

Dalším krokem byl vývoj skriptu pro měření serverové odezvy, kde jsem rovněž aplikoval principy multivláknového zpracování. S kompletním souborem dat jsem následně navrhl API, které umožňovalo interakci se skripty, a strukturoval databázi pro ukládání informací.

V rámci další fáze jsem se zaměřil na vývoj uživatelského rozhraní, kde jsem se věnoval zpracování dat a implementaci funkcí potřebných pro frontend aplikace. Po dokončení těchto kroků jsem přistoupil k integraci grafů do webového rozhraní, což byl poslední krok před zahájením testovací fáze.

Aplikaci jsem nasadil na virtualizovaném pracovišti a po několika úpravách začala fungovat bez problémů. Během testování se však objevily problémy s výpadky dat z Apache statistik, toto se ale ani přes velkou snahu nepodařilo opravit.

5.2 Sběr dat pomocí Python skriptů

Dle zadání byly vytvořeny dva skripty v programovacím jazyce Python. První skript slouží k sběru dat ze serveru a druhý k monitoringu odezvy. Oba skripty běží nezávisle na uživateli a je možné je spustit automaticky. Základní konfigurace těchto skriptů je načítána z konfiguračního souboru, který se v případě absence automaticky generuje v adresáři, z něhož byl skript spuštěn. V konfiguračním souboru je také nastaven parametr `run_enable` s výchozí hodnotou `False`, což vyžaduje, aby uživatel tento soubor upravil pro spuštění měření. Detailnější nastavení skriptů je následně definováno v konfiguračním *JavaScript Object Notation* (JSON) souboru, jenž je stahován během pokusu o připojení k webové aplikaci.

5.2.1 Monitoring serverových prostředků

Skript `server_script.py` je navržen pro sběr dat o zatížení hardwaru a využití serverových prostředků. Jeho základní konfigurace je definována čtyřmi parametry, které jsou podrobně popsány v tabulce 5.1.

Tab. 5.1: Popis konfiguračních parametrů souboru `server_script.ini`

Parametr	Funkce	Hodnota
<code>web_server_ip</code>	IP adresa webové aplikace	-
<code>registration_token</code>	Token vygenerovaný webovou aplikací	6místný identifikátor
<code>apache_interface</code>	Název sledovaného rozhraní	-
<code>run_enable</code>	Potvrzení pro běh skriptu	True/False

Skript využívá různé knihovny k získání informací o zatížení CPU, využití paměti RAM a SWAP, využití diskového prostoru a rychlosti čtení a zápisu. Data z síťových rozhraní nám poskytují informace o množství a stavu spojení filtrovaných pro konkrétní port nebo o rychlosti přenosu dat podle zvoleného síťového rozhraní. Díky vícevláknovému designu lze tyto úlohy spustit periodicky v definovaných intervalech, aniž by bylo nutné čekat na dokončení předchozího měření.

Pro získání dat z Apache serveru se ukázalo jako nejefektivnější využít stránku `/server-status?auto`. Analýza této stránky nám umožňuje extrahovat informace o počtu požadavků za sekundu, době potřebné k zpracování odpovědi, a stavu jednotlivých workerů. V případě, že Apache server neodpovídá a není možné načíst stránku s informacemi, je nastavený timeout 3 sekundy. Po jeho uplynutí je požadavek ukončen a s ním i příslušné vlákno. Tato funkcionality slouží jako ochrana proti nadměrnému vytváření vláken, která by čekala na odpověď. V případě výpadku nebo nedostupnosti Apache serveru jsou uživatelům poskytnuta alespoň ostatní data. Stanovená doba pro timeout je dostatečně dlouhá, aby pokryla eventuální zpoždění, avšak není natolik prodloužená, aby zbytečně zdržovala odeslání ostatních statistik.

Bohužel se mi nepodařilo nastavit Apache server tak, aby i při DoS útoku byla data dostupná. Pro získání dat jsem zkoušel několik metod, včetně logování, použití příkazové řádky s nástrojem Links a pokusy o rezervaci workerů pro obsluhu skriptových požadavků. Ani jedna z těchto metod nevedla k úspěšnému získání adekvátních dat během DoS útoku. Tato zkušenost poukazuje na výzvy spojené s monitorováním serveru v extrémních podmínkách a potvrzuje potřebu dalšího výzkumu v této oblasti.

Tento skript rovněž poskytuje funkci, která po ověření spojení s webovou aplikací automaticky odesílá informace o konfiguraci serveru. Tím je uživateli umožněno získat podrobnější přehled o monitorovaném serveru. Na webovém rozhraní je možné

následně zobrazit různé údaje o serveru, jako je počet jader procesoru, jeho typ, velikost operační paměti RAM, počet síťových rozhraní, nebo verzi nainstalovaného Apache serveru.

5.2.2 Měření odezvy serveru

Druhý ze skriptů, `network_script.py`, je zaměřen na měření doby odezvy serveru. Tento skript má méně konfiguračních parametrů, jejich vysvětlení nabízí tabulka 5.2. I zde se využívá vícevláknová architektura, což umožňuje periodické odesílání požadavků na testovaný webový server Apache a sledování doby jeho reakce. Díky implementaci vícevláknového zpracování jsou požadavky odesílány kontinuálně. Tento mechanismus je zvláště relevantní v případech, kdy může dojít k odepření služby v důsledku DoS útoku a schopnost stále vytvářet nové requesty v pravidelných intervalech je klíčová.

Tab. 5.2: Popis konfiguračních parametrů souboru `network_script.ini`

Parametr	Funkce	Hodnota
<code>web_server_ip</code>	IP adresa webové aplikace	-
<code>run_enable</code>	Potvrzení pro běh skriptu	True/False

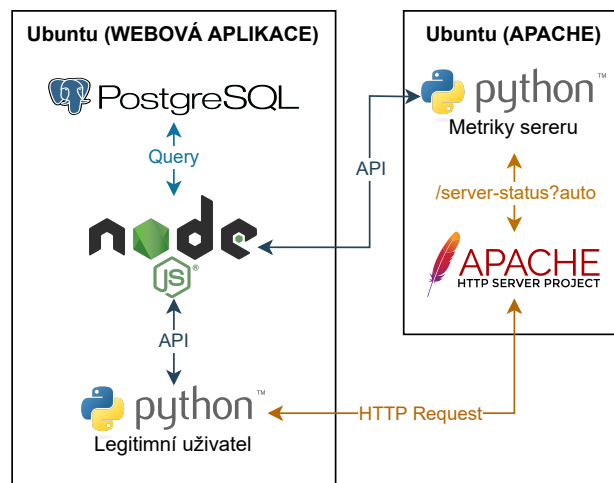
Pro správnou funkci bylo nutné nastavit timeout po jehož vypršení se požadavek ukončí a odešle se doba trvání s hodnotou `null`. Server se tak následně bude v aplikaci jevit jako nedostupný. Při stanovení konstanty pro timeout v měření času odezvy serveru bylo zvoleno 12 sekund, což odráží kompromis mezi nutností zajistit realistické měření doby odezvy a potřebou minimalizovat náklady spojené s čekáním na odpověď serveru. Tento výběr byl založen na několika klíčových faktorech:

- **Délka odezvy na první požadavek:** Naměřená data ukazují, že doba odezvy prvního requestu na Apache server umístěném na HDD se pohybuje mezi 6,1 a 9,5 sekundami. Tento rozsah naznačuje, že v některých případech může být čas načtení delší, než standardní timeouty při běžném provozu webového serveru.
- **Uživatelská tolerance k načítání stránky:** Výzkum na webu Rychlost.cz uvádí, že 40% návštěvníků opustí stránku, pokud se není schopna načíst během 3 sekund [7]. Zvolený timeout 12 sekund tedy poskytuje dostatečnou rezervu nad rámec této kritické hranice, aby bylo možné zachytit i pomalejší odezvy bez předčasného ukončení měření.
- **Rezerva pro krátkodobé výpadky:** Delší timeout umožňuje absorpci krátkodobých výkyvů v odezvě serveru, které mohou být způsobeny dočasným přetížením serveru nebo zpožděními v síťovém přenosu. Tím je zajištěno, že

měření poskytne spolehlivější údaje o skutečné dostupnosti a výkonnosti serveru. Navíc, tato rezerva minimalizuje riziko falešně pozitivních výsledků, které by mohly vzniknout z přechodných problémů, a poskytuje tak přesnější obraz o stabilitě a spolehlivosti serveru.

5.3 Zpracování dat a webová aplikace

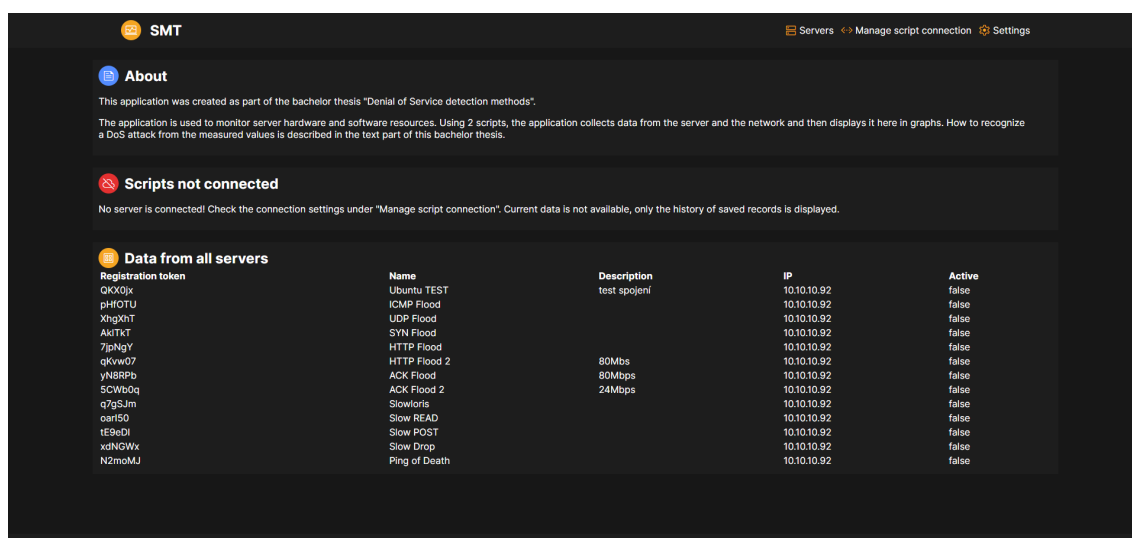
Data ze skriptů se do webové aplikace posílají pomocí *Application Programming Interface* (API). Přijatá data v JSON formátu jsou následně uložena pomocí query do připravených tabulek v připojené databázi. Jelikož vyhledávání v databázi neprobíhá na základě metrik, jsou data uložena do tabulky v JSON formátu, což značně zvýšilo jednoduchost implementace.



Obr. 5.1: Schema fungování aplikace

Samotné webové rozhraní této aplikace nabízí několik stránek.

- **Úvodní stránka:** Zde najdeme sekci s aktuálními informacemi pro aktivní token a tabulku s přehledem registrovaných serverů, která funguje jako proklik na detailní zobrazení informací a metrik. Informace o tokenu slouží jako možnost kontroly zda oba skripty odesílají data.
- **Správa připojení skriptu:** Tato stránka umožňuje registraci nového serveru (měření), aktivaci již existujícího tokenu (měření) a v případě že aktuálně probíhá sběr dat, můžeme zde token deaktivovat a pozastavit tak skripty.
- **Nastavení:** V případě potřeby změny chování webového rozhraní můžeme na této stránce najít užitečné funkce.



Obr. 5.2: Úvodní stránka webové aplikace

Zoom mode nabízí zvětšení grafů na stránce s informacemi o serveru na celou šířku stránky a získáme tak větší podrobnost grafu.

Refresh pause definuje periodu načítání nových dat pro grafy.

Number of displayed values ovlivňuje SQL požadavek a mění počet vrácených záznamů pro zobrazení.

Server port upravuje filtrování provozu serverového scriptu. Při změně tohoto parametru je ale nutné zastavit a znovu spustit měření, aby se port změnil!

Posledním parametrem, který lze změnit je Auto update. Tato funkce zapíná a vypíná automatické načítání dat pro grafy.

- **Detail serveru:** Na tuto stránku se dostaneme přes tabulku serverů na úvodní stránce a kliknutím na požadovaný server. Tato stránka se vykresluje dynamicky na základě zvoleného tokenu a dělí se na 2 sekce. V první části jsou zobrazeny informace o serveru, období ze kterého pocházejí vykreslená data v grafech a tlačítko pro export dat v CSV formátu. To je vhodné zejména v případech, že chceme zobrazit zakyslosti, které následující grafy nenabízí, nebo ve chvíli kdy jedna hodnota zkresluje měření natolik, že jsou grafy nečitelné. V druhé části se nachází již zmiňované grafy. Ty nabízí pohled na veličiny v závislosti na jednotné časové ose a možnost skrývání jednotlivých datasetů.

5.4 Fungování aplikace jako celku

Celá aplikace funguje na principu tokenu, který umožňuje jednoznačnou identifikaci přijatých dat. I když systém následně nahradí 6místný token identifikátorem serveru v registrační tabulce, primární úlohou tokenu je zajištění ochrany proti chybám

při zadávání. Využití pouze jednoho čísla nebo písmene pro registraci serveru by mohlo vést k omylem zapsaným datům pro nesprávný server. Tokeny také umožňují spravovat více registrací současně, což rozšiřuje možnosti ukládání naměřených dat.

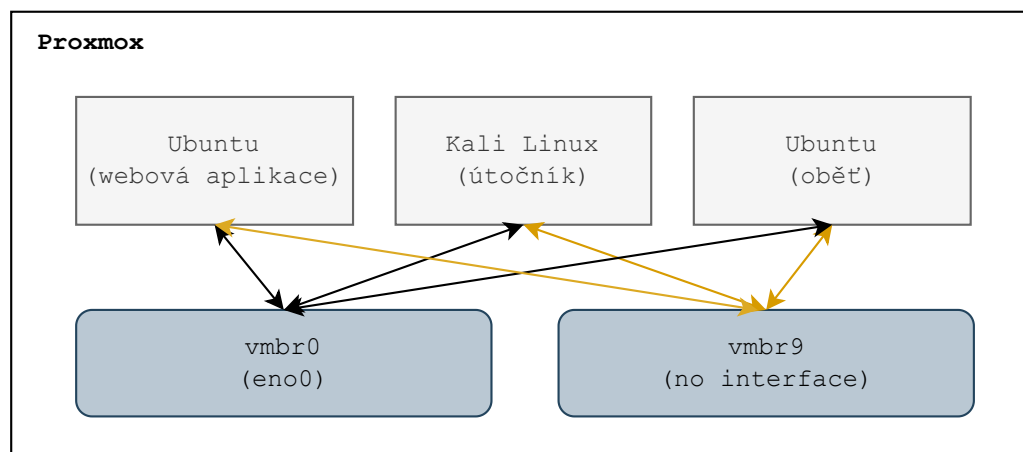
Když je token jednou vygenerován a po provedení měření deaktivován, data zaznamenaná během jeho platnosti zůstávají dostupná pro zobrazení. Tento přístup umožňuje porovnávat mezi sebou různá nastavení a dopady útoků.

Je však nutné dbát na správnou konfiguraci serverového skriptu, především na aktualizaci tokenu. Pokud by v konfiguraci zůstal starý token, mohlo by to vést k potenciální ztrátě dat nebo nesprávnému zaznamenávání informací.

5.5 Představení virtualizovaného pracoviště

Pro účely virtualizace jsem zvolil server s platformou Proxmox, přičemž jsem využil stávající hardware, konkrétně server Dell PowerEdge R720 v2.

V rámci tohoto prostředí jsem použil existující virtuální bridge `vmbr0`, který je prostřednictvím fyzického portu `eno0` připojen k síti serverů. Díky *Virtual Private Network* (VPN) mám možnost vzdáleného přístupu k této síti, což zajišťuje bezpečnou a šifrovanou komunikaci a dostupnost mého pracoviště kdekoliv. Navíc jsem vytvořil druhý virtuální bridge `vmbr9`, který je specificky určen pro simulaci komunikace mezi útočníkem a obětí. Použití více síťových karet pro jedno virtuální zařízení bylo záměrné, aby i při plném využití připojení na `vmbr9` během DoS útoku, bylo stále možné bez problémů získávat aktuální hodnoty pro webovou aplikaci.



Obr. 5.3: Virtualizace strojů pomocí Proxmox

Tab. 5.3: Přehled IP adres dle rozhraní.

Název stroje	IP adresa	
	vmbr0	vmbr9
web	10.0.0.90	10.10.10.90
útočník	10.0.0.91	10.10.10.91
oběť	10.0.0.92	10.10.10.92

5.5.1 Virtuální stroj oběti

Pro virtuální stroj oběti byly zvoleny následující hardwarové specifikace: 4 GB RAM paměti, 4 jádra procesoru typu kvm64 a SSD disk o kapacitě 32 GB sloužící jako úložiště. Obě síťová rozhraní, která jsou připojena k tomuto virtuálnímu stroji, mají v základní konfiguraci rychlost 1 Gbps.

Na tomto virtuálním stroji je nainstalován operační systém Ubuntu 22.04.1. Jako webový server je zde použit Apache ve verzi 2.4.52. Tato kombinace operačního systému a webového serveru byla vybrána z důvodu jejich široké podpory, stabilní výkonosti a rozsáhlé dokumentace, která usnadňuje správu a konfiguraci pro účely testování.

Konfigurace hardwaru a softwaru byla zvolena tak, aby odpovídala menšímu webovému serveru s návštěvností okolo 100 klientů denně, což umožňuje realistické testování a analýzu útoků, protože s větším výkonem serveru by bylo těžší docílit stavu odepření služby. SSD disk byl zvolen pro svou vysokou rychlost čtení a zápisu, což minimalizuje úzká místa způsobená diskovým I/O a umožňuje zaměřit se na analýzu síťových a procesorových nároků během testování.

Virtuální stroj je připojen k síti pomocí dvou gigabitových síťových rozhraní, což poskytuje možnost nezávislého připojení pro odesílání metrik i v případě DoS útoku na jednom rozhraní. Tato konfigurace umožňuje důkladné testování zátěže a analýzu síťového provozu. Monitorovací skript je současně nastaven tak, aby komunikaci na tomto rozhraní určeném pro služební komunikaci ignoroval a nebyly tak zkreslené naměřené hodnoty.

6 Určení metod detekce DoS

Pro detekci DoS lze využít sofistikované technologie, jako jsou *Intrusion Detection Systems* (IDS) a *Intrusion Prevention Systems* (IPS), které kombinují různé metody monitorování a analýzy dat k identifikaci podezřelého chování v síti.

IDS pracují primárně na principu detekce podezřelých aktivit v síťovém provozu. IDS může být implementován jako síťový *Network intrusion detection system* (NIDS), který sleduje data procházející skrz celou síť, nebo jako hostitelský *Host-based intrusion detection system* (HIDS), který monitoruje aktivity na konkrétním serveru nebo zařízení. Dále se využívají pokročilé analytické nástroje a technologie strojového učení pro dynamické hodnocení síťového chování a jeho porovnávání s dlouhodobými trendy a historickými daty.

IPS jsou navrženy tak, aby nejen detekovaly útoky, ale také aktivně zasahovaly proti nim v reálném čase. IPS systémy jsou obvykle integrovány přímo do síťové infrastruktury a mohou automaticky blokovat podezřelý provoz, izolovat zasažené systémy, nebo upravit firewall pravidla, aby ochránily síť. IPS obvykle využívá stejné detekční techniky jako IDS, ale s tím rozdílem, že má schopnost přímo reagovat na detekované hrozby. [6]

6.1 Detekce založená na signaturách

IDS používají metodu založenou na signaturách, kde se monitorovaný provoz porovnává s databází známých útočných vzorů nebo „signatur“. Tento způsob je velmi účinný proti hrozbám, které byly již dříve identifikovány, popsány a katalogizovány. Avšak, hlavní omezení tohoto způsobu spočívá v neschopnosti rozpoznávat nové nebo upravené typy útoků, které se významně liší od již známých vzorů. Tím pádem, pokud útočníci modifikují své metody nebo použijí doposud neznámé taktiky, může dojít k selhání této detekční strategie. Současně musí tato strategie počítat s běžným provozem a důkladně proto ověřovat nově přidávané vzorce chování, aby nedocházelo k falešným identifikacím. [25]

6.2 Detekce založená na anomáliích

Tento přístup k detekci DoS útoků se opírá o analýzu běžného provozního chování a identifikaci odchylek, které mohou signalizovat možný útok. Anomální chování může zahrnovat různé indikátory, jako je neočekávaně vysoký počet požadavků z jedné zdrojové IP adresy, neobvyklé vzorce provozu, jako jsou nezvykle velké objemy dat odeslané v krátkém časovém úseku, nebo neobvyklé časy aktivit. Tento model se neustále učí a adaptuje na základě kontinuálně sbíraných dat, což umožňuje

detekovat i předtím neviděné typy útoků. Nicméně, výzvou pro tento typ detekce je možnost falešně pozitivních hlášení, kdy běžné, avšak neobvyklé aktivity mohou být nesprávně identifikovány jako útočné. [25]

6.3 Falešně pozitivní výsledky detekce

I přes pokročilé bezpečnostní technologie, jako jsou IDS a IPS, se organizace často setkávají s výzvou falešně pozitivních výsledků. Tato chybná identifikace může vést k narušení legitimního provozu a zbytečným administrativním zatížením. Pochopení příčin falešně pozitivních výsledků a implementace efektivních strategií pro jejich minimalizaci je základním prvkem zlepšení efektivity bezpečnostních opatření. [22]

Mezi hlavní příčiny falešně pozitivních výsledků patří:

- **Příliš restriktivní rate limiting:** Rate limiting je běžně používán k omezení počtu požadavků, které uživatel může zaslat na server za určité časové období, aby se zabránilo vyčerpání zdrojů serveru. Avšak, pokud jsou limity nastaveny příliš striktně, může to vést k nesprávnému klasifikování normálního zvýšení provozu jako útočného chování, zvláště během vrcholných období nebo marketingových kampaní.
- **Chybná konfigurace geo blokace:** Geo blokace se využívá k omezení přístupu z určitých geografických oblastí jako opatření proti kybernetickým útokům. Tato technika může neúmyslně zablokovat legitimní uživatele, pokud je zeměpisná identifikace IP adres nepřesná nebo pokud dojde k nesprávnému označení regionů jako zdroje útoků.
- **Nesprávně nastavený firewall:** Firewally jsou zásadní pro obranu sítě, ale jejich příliš restriktivní pravidla mohou omezovat platný provoz. Například blokování specifických portů nebo protokolů může nechtěně omezit služby, které používají tyto porty pro legitimní účely.
- **Omezení při konfiguraci serverů:** Preventivní konfigurace, jako jsou bezpečnostní aktualizace a omezení přístupu, jsou důležité pro zabezpečení, ale mohou způsobit problémy, když jsou nesprávně implementovány. Například, přísná pravidla pro autentizaci a autorizaci mohou vyvolat falešné alarmy v situacích, které jsou ve skutečnosti normální operace uživatele.

Strategie pro minimalizaci falešně pozitivních výsledků při detekci hrozeb zahrnují několik klíčových opatření. Prvním z nich je pravidelná revize a aktualizace bezpečnostních nastavení. Bezpečnostní týmy by měly systematicky revidovat a aktualizovat pravidla pro rate limiting, nastavení firewallů a politiky geo blokace. Toto opatření zahrnuje kalibraci prahových hodnot, které by měly být nastaveny na základě nejnovějších dat o provozu a identifikovaných hrozbách, aby byly co nejpřesnější.

Druhým opatřením je implementace adaptivních bezpečnostních systémů. Tyto systémy, které využívají principů strojového učení a umělé inteligence, se automaticky přizpůsobují měnícím se podmínkám sítě a učí se z historických dat. Díky tomu jsou schopny efektivněji rozlišovat mezi běžnými vzory provozu a potenciálními bezpečnostními hrozbami, čímž se minimalizuje riziko falešně pozitivních detekcí.

Třetím klíčovým prvkem je monitoring a analýza. Rozsáhlé sledování a detailní analýza širokého spektra metrik mohou poskytnout hlubší vhledy do charakteru provozu a pomoci odlišit skutečné hrozby od běžných operací. Kromě monitorování objemu provozu by měla analýza zahrnovat i hodnocení typů provozu, délky spojení a chování uživatelů, což vše napomáhá k přesnější identifikaci útoků. Tato strategie přispívá k robustnější a přesnější obraně proti potenciálním hrozbám. [22]

6.4 Detekce pomocí vlastní aplikace

Detekce DoS útoků lze efektivně provádět sledováním hardwarových a softwarových prostředků serveru. Záplavové útoky, charakteristické svou schopností kompletně zaplnit síťové připojení, jsou obvykle snadněji identifikovatelné. Menší servery jsou zvláště náchylné k tomu, že dříve dosáhnou limitů své šířky pásma připojení než plného využití hardwarových kapacit. Útoky cílící na aplikační vrstvu modelu TCP/IP se projevují vysokým počtem aktivních spojení. Tento druh útoku lze monitorovat skrze sledování anomálních vzorců v počtu spojení.

V rámci této práce byla vyvinuta aplikace schopná monitorovat prostředky jak na straně serveru, tak na straně sítě, přičemž výsledky jsou zobrazeny ve webové aplikaci. Tato aplikace poskytuje uživatelům nástroj pro sledování aktuálních i historických dat, která mohou být využita k detekci odchylek od standardního provozu. Různé typy útoků se manifestují odlišnými vzorci chování, což může být na zobrazených datech patrné.

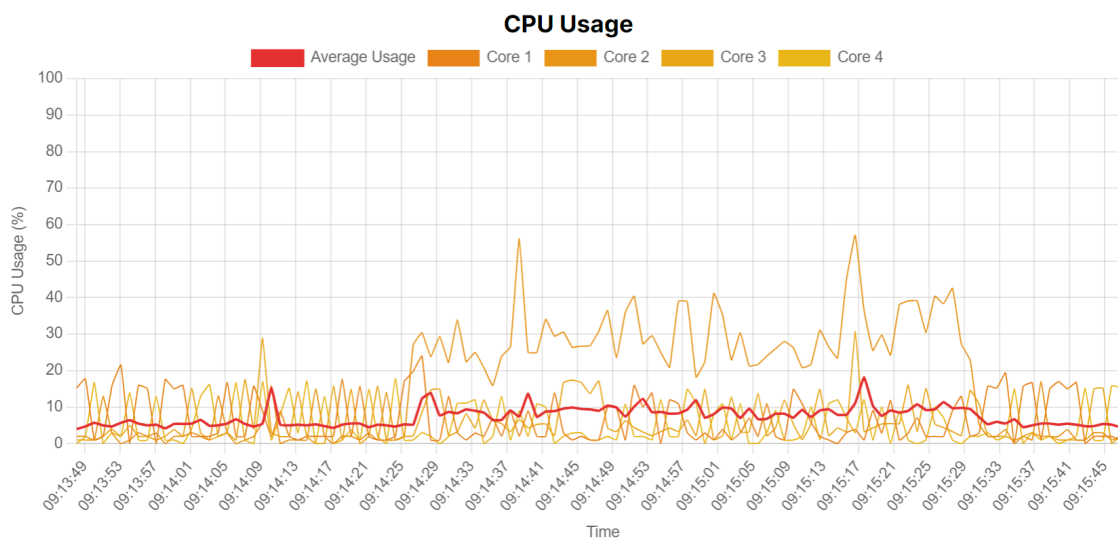
Základním ukazatelem nám však může být graf ukazující odezvu serveru. Pokud nastane okamžik že mají data hodnotu 12 s a jsou podbarvena červeně můžeme jednoznačně říct že došlo k odepření služby legitimnímu uživateli. Tento stav ale nutně neznamená útok. K tomuto stavu může například dojít i v případě, že připojení serveru není dostatečně výkonné, nebo samotný server není připraven na vysoké vytížení ze strany klientů.

Jak ale spolehlivě poznáme DoS? Pojďme se teď podívat na to, jak se jaký útok chová a na postup jak útok identifikovat. Všechny grafy v popisu chování jednotlivých útoků jsou staženy z webové aplikace pomocí rozšíření pro prohlížeč **Find elements diff** a upraveny na bílé pozadí z důvodu tisku. Cílem každého útoku bylo dosáhnout největšího možného zatížení dostupných prostředků při době trvání okolo 60 vteřin. Kompletní podklady k testování jsou součástí elektronické přílohy.

ICMP Flood

Při testování tohoto útoku bylo počítáno s jeho záplavovým efektem a proto byla nastavena maximální propustnost virtuálního síťového připojení na 120 Mbps. Přičemž maximální generovatelný datový tok byl o 23 Mbps větší. Ke generování byl použit nástroj Hping3 ve verzi 3.0.0-alpha-2.

Následek tohoto útoku se projevil pouze na množství dat, které protekly přes síťovou kartu, drobným nárůstem zatížení CPU a zvýšením odezvy serveru z jednotek ms až na 7 404 ms. Ovšem k překročení kritické doby odezvy v podobě 12 s nedošlo a aplikace tak zobrazovala Apache server jako dostupný po celou dobu trvání útoku.

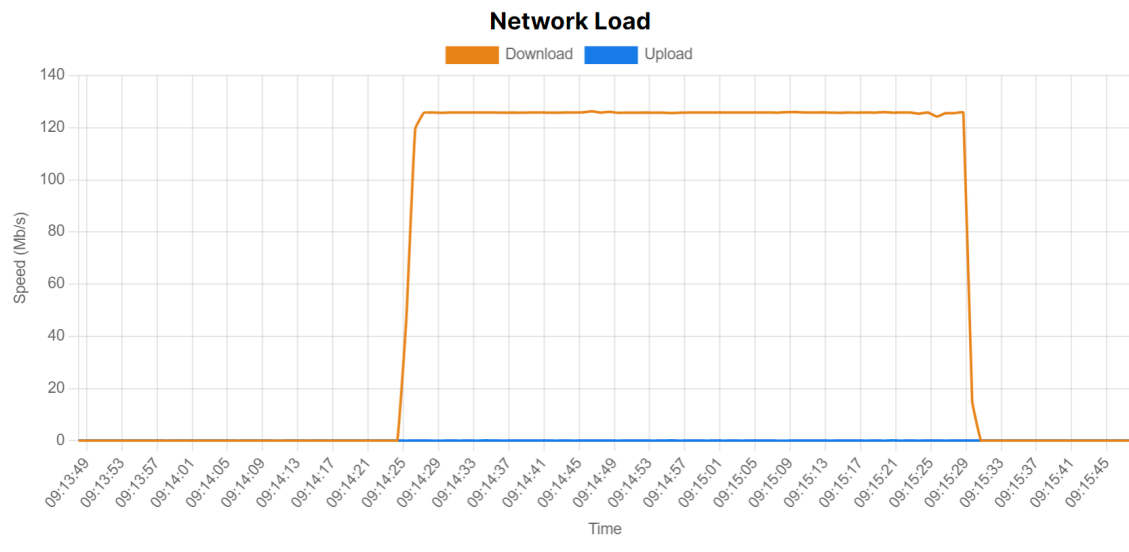


Obr. 6.1: Změna zatížení jádra v průběhu ICMP Flood útoku

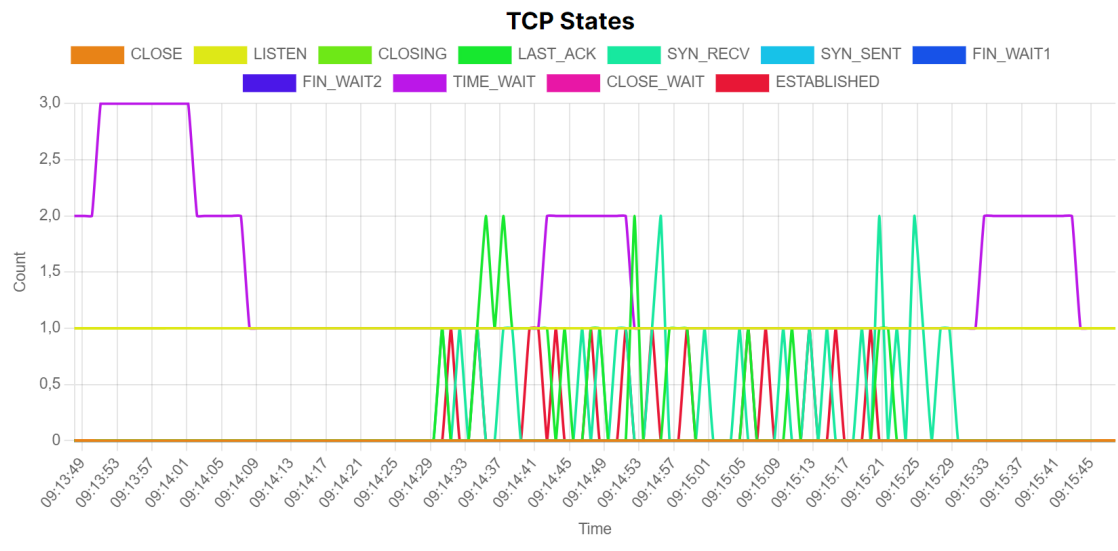
Při analýze síťového provozu, kde datový tok dosahuje 125 Mbps a současně prochází 41 176 paketů za sekundu, lze z těchto údajů vyvodit několik závěrů. Tento stav může být indikátorem neobvyklého chování v síti a může být spojen s potenciálním DoS útokem, avšak není možné okamžitě a jednoznačně potvrdit DoS útok bez dalších informací. Samotné dosahování maximálního datového toku 125 Mbps znamená, že síťové rozhraní je plně využito a operuje na své kapacitní hranici. Toto je běžné během špičkového provozu, například při stahování nebo nahrávání velkých souborů, streamování videa ve vysoké kvalitě nebo při zálohování dat.

Tato data v kombinaci s konstantním množstvím paketů za sekundu ukazují, že síť přenáší velké množství malých paketů. Takové chování může být neobvyklé, pokud se nevyskytuje pravidelně a bez specifického důvodu. Obvyklý síťový provoz zahrnuje větší pakety, zejména při přenosu souborů nebo streamování, nebo jejich různou nekonzistentní velikost.

Data zahrnující softwarové prostředky Apache nejsou v tomto případě pro použití při detekci adekvátní, protože ICMP Flood necílí na zatížení webového serveru. Ani graf 6.3 zobrazující aktivní TCP spojení na portu 80 nenese žádné známky útoku.



Obr. 6.2: Nárůst datového toku internetového rozhraní v průběhu ICMP Flood útoku

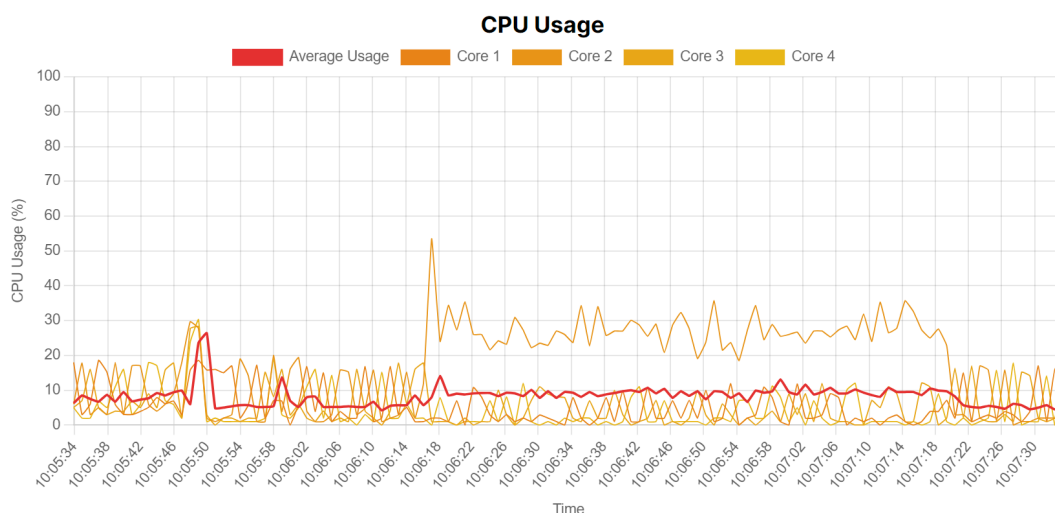


Obr. 6.3: Aktivní TCP spojení při ICMP Flood útoku

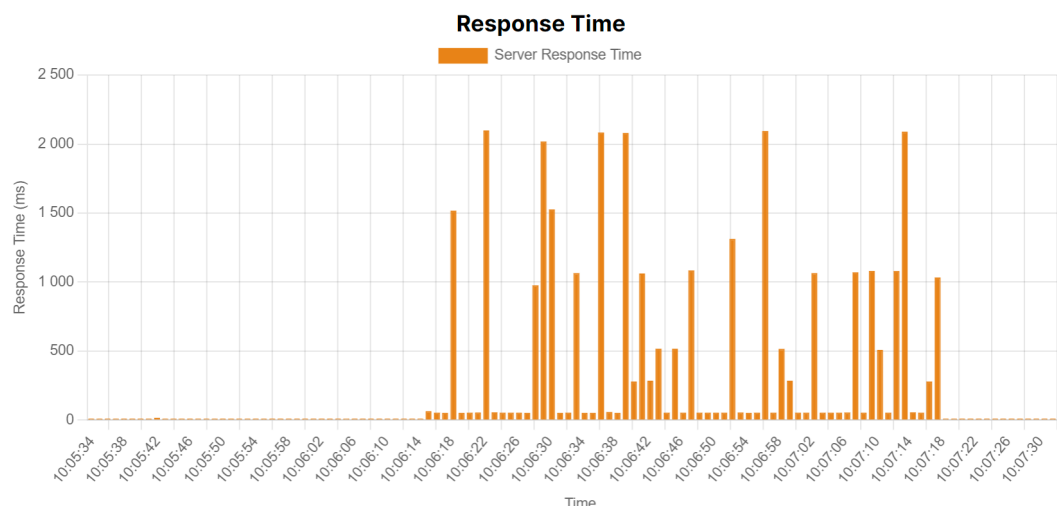
UDP Flood

I tento útok byl generován nástrojem Hping3 a byla zachována i maximální přenosová rychlost virtualizovaného připojení oběti na 120 Mbps. Útok byl směřován na port 80 s co největší možnou intenzitou. Toho u Hping3 docílíme přepínačem `-flood`.

Reakce systému na tento útok byli velmi podobné jako v předchozím případě útoku ICMP Flood. Pro porovnání zátěže útoku na procesor slouží graf 6.4. Opět se setkáváme také s vysokým datovým tokem malých paketů s konstantní velikostí. Jedinou změnou byla doba odezvy webového serveru, ta oproti předchozímu testu stoupla na pouhých 2 099 ms. Celý průběh odezvy je zobrazen na grafu 6.5.



Obr. 6.4: Nárůst zatížení procesoru důsledkem útoku UDP Flood



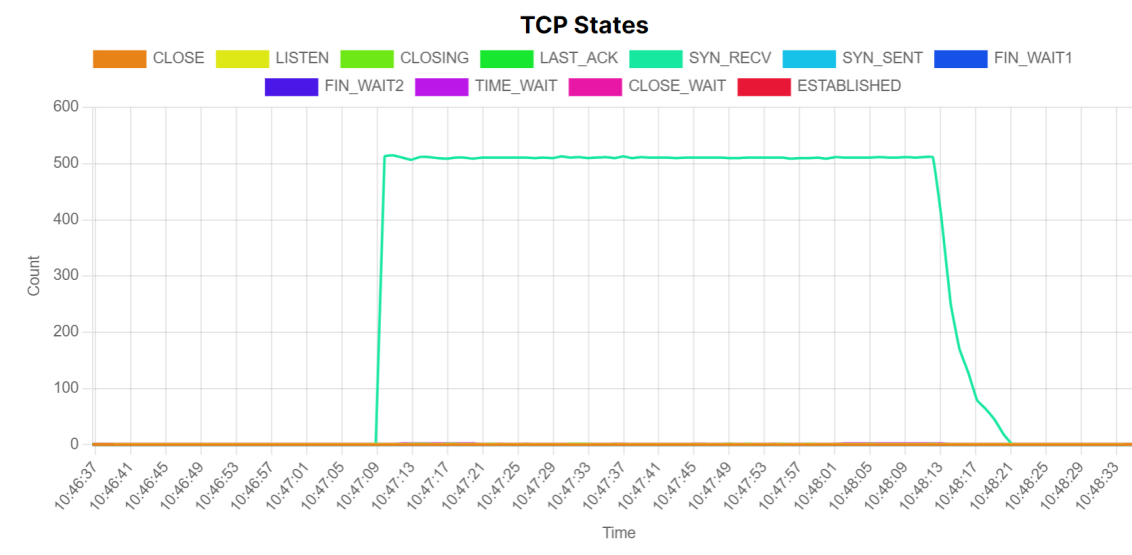
Obr. 6.5: Zvýšení doby odezvy při útoku UDP Flood

SYN Flood

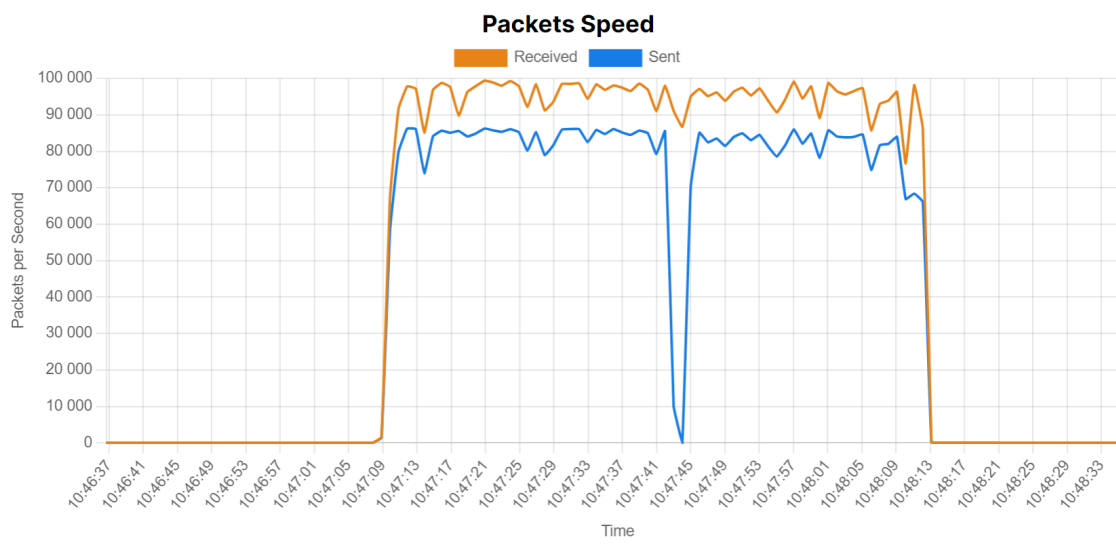
Při testování SYN flood útoku byly zaznamenány odlišné výsledky ve srovnání s předchozími útoky. Při testování byl zvýšen maximální datový tok virtuálního připojení napadeného serveru na 240 Mbps, což umožnilo přijímat větší objem dat. Jelikož používaný **Hping3** již nestačil, byl využit **Trafgen** verze 0.6.8. Tento nástroj umožnil vytvoření většího množství paketů s vysokou rychlostí, což zvýšilo celkový datový tok a zátěž sítě.

Zvýšený objem dat je přímým důsledkem intenzity SYN flood útoku. Vysoká rychlost připojení umožnila útočníkovi generovat a odesílat SYN pakety rychleji, což vedlo k vyššímu zatížení síťového rozhraní serveru. Jelikož server reaguje na každý příchozí SYN paket SYN_ACK paketem, což je standardní součást TCP handshake, vede tento proces k dalšímu nárůstu zátěže CPU, protože server musí generovat a odesílat odpovědi na každý falešný požadavek. Graf 6.7 jasně ukazuje tuto interakci, kde zvýšený počet příchozích SYN paketů odpovídá zvýšenému počtu odchozích SYN-ACK paketů.

Během útoku bylo zaznamenáno celkem 517 aktivních soketů, přičemž výrazná většina byla ve stavu SYN_RECV. Tento stav je charakteristický pro SYN flood útoky a značí, že server očekává dokončení TCP handshake procesu, který však nikdy nenastane. Popisovanou situaci ilustruje graf 6.6, kde můžeme vidět jednotlivé stavy TCP spojení na portu 80 v době útoku.



Obr. 6.6: Stav TCP soketů při útoku SYN Flood



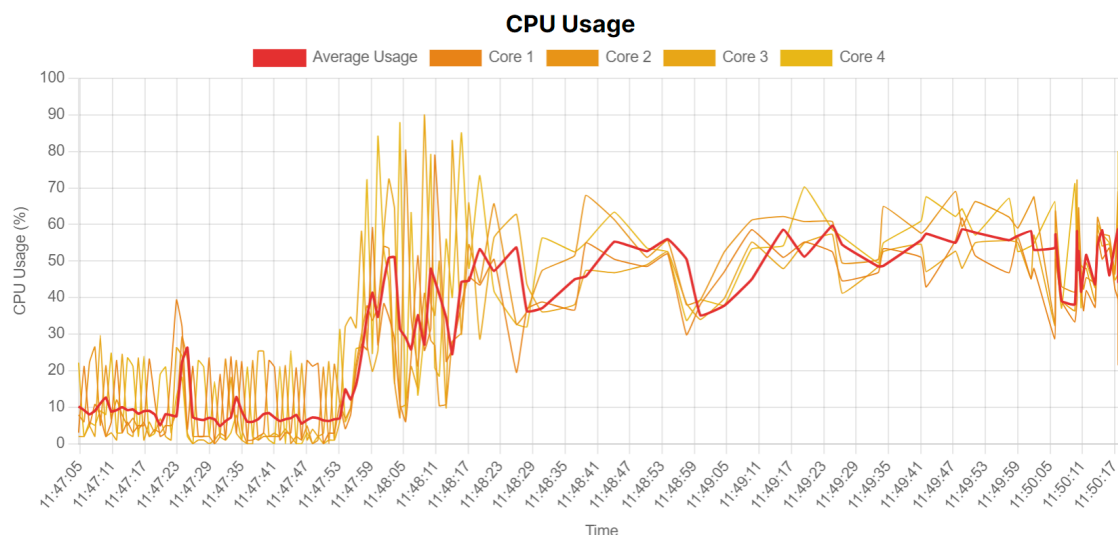
Obr. 6.7: Množství přijatých a odeslaných paketů při testování SYN Flood

HTTP Flood

Tento útok byl proveden pomocí nástroje Apache HTTP server benchmarking tool, známého také jako **ab**. Tento nástroj je primárně určen k měření výkonu webových serverů a umožňuje generovat velké množství HTTP požadavků s cílem testovat, jak server zvládá zátěž. Může ale být zneužit k provedení HTTP Flood útoků. V tomto konkrétním případě bylo na server odesláno 50 000 požadavků s tím, že se testovací nástroj snažil držet 500 požadavků současně. Celý tento provoz server odbavil za zaokrouhleně 77 vteřin, což je daleko nad rámec námi definovaného standardního provozu.

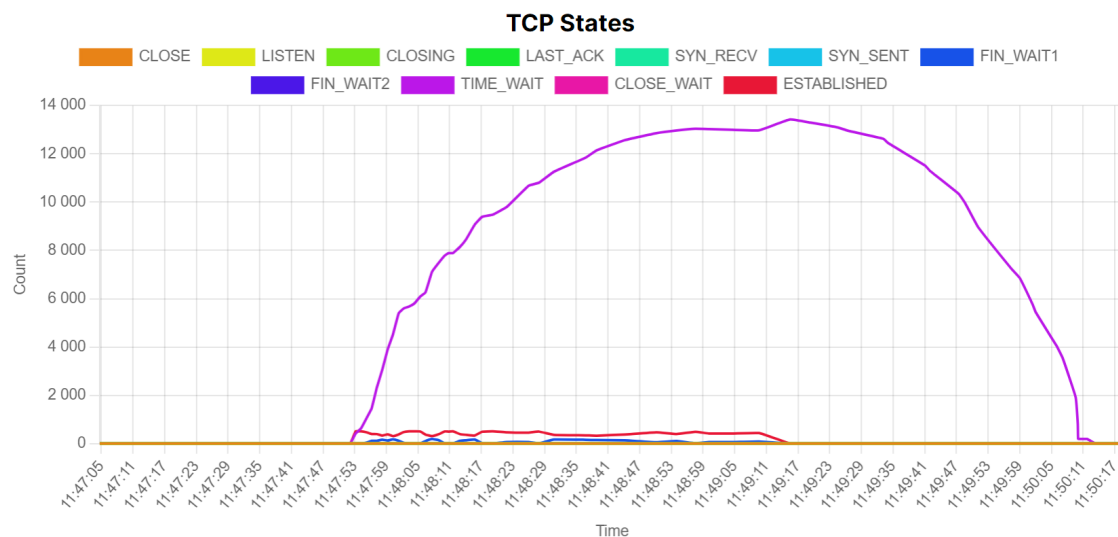
Z měření vyplývá, že zatížení CPU během HTTP Flood útoku bylo výrazně větší v porovnání s předchozími útoky. Tento typ útoku totiž vyžaduje od serveru zpracování velkého množství HTTP požadavků, což znamená značné množství výpočetních operací. Každý přijatý požadavek musí být serverem zpracován a odpověď musí být odeslána zpět klientovi. Toto zatížení se výrazně projevilo na celkovém výkonu serveru, který přestal zvládat nápor požadavků a došlo dokonce k výpadkům monitorovacího skriptu. Kvůli zmíněné vysoké zátěži CPU a následným výpadkům bylo nezbytné omezit rychlost virtuálního připojení přes které byl veden útok na 120 Mbps. Toto omezení pomohlo stabilizovat server a umožnilo pokračování monitorování a analýzy útoku s menšími výpadky. Omezení rychlosti připojení je běžnou obranou strategií, která může snížit dopad útoku a umožnit serveru zotavení.

Za pozornost stojí také fakt, že upload serveru je výrazně vyšší než jeho download, útočník může s relativně malým objemem odeslaných požadavků vytvořit velký objem odpovědí, které mají větší velikost. Pokud by na trase mezi útočníkem a cí-



Obr. 6.8: Výrazné zatížení CPU při útoku HTTP Flood

lovým serverem byl router s omezenou routerovací kapacitou, dopady HTTP Flood útoku by mohly být ještě závažnější, neboť větší velikost odpovědí ještě zvýší zatížení routeru a celkové sítě. Tato situace zvyšuje efektivitu útoku a způsobuje větší narušení služeb při menším úsilí ze strany útočníka. Popisované navýšení objemu odchozích dat znázorňuje graf 6.10.

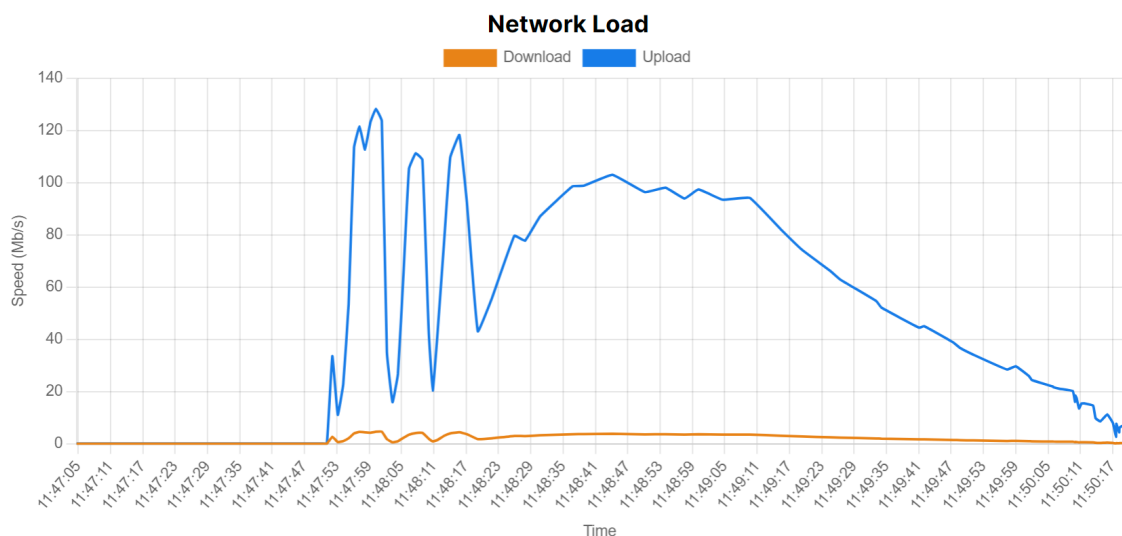


Obr. 6.9: Graf TCP stavů na portu 80 při útoku HTTP Flood

Dalším významným dopadem HTTP Flood útoku bylo vytvoření 14 503 spojení ve stavu `TIME_WAIT`. Tento stav je výsledkem uzavírání TCP spojení, kdy server čeká, zda nedojde k retransmisi posledního ACK paketu. Vysoký počet spojení ve

stavu `TIME_WAIT` indikuje, že server musel zpracovávat velké množství ukončených spojení, což dále zatěžovalo jeho zdroje. Důležité je poznamenat, že tvorba těchto připojení nebyla rázová, nýbrž postupná. To naznačuje, že útok byl navržen tak, aby průběžně generoval požadavky, čímž se vyhnul rychlému odhalení a obcházel jednoduché obranné mechanismy, které jsou schopny zvládat krátkodobé nápor.

HTTP Flood útok specificky cílí na aplikační vrstvu, což se projevilo na statistikách Apache serveru. Samotný server měl v tuto dobu k dispozici 200 workerů a po celou dobu zvládal požadavky plnit. V nejvyšším bodě útoku ukazovala Apache statistika `Requests Per Second` 175 požadavků za vteřinu. Tato statistika ale ukazuje průměr období a není schopna tak reagovat na špičkové změny. Ve stavu, kdy dlouho na server neprobíhaly žádné útoky se tato hodnota pohybovala na úrovni 1 požadavek za sekundu. Na začátku tohoto testu již ale byla na 110 požadavcích a měla klesající tendenci. Bohužel tato skutečnost byla zapříčiněna předchozím testem. Reálný počet požadavků tak můžeme vyčíst z testovacího nástroje, kde se dozvíme že průměr byl 649 požadavků za sekundu. Tento počet výrazně převyšuje běžný provoz a vede téměř k vyčerpání prostředků serveru. Při testování více souběžných spojení již došlo ke stavu odepření DoS.



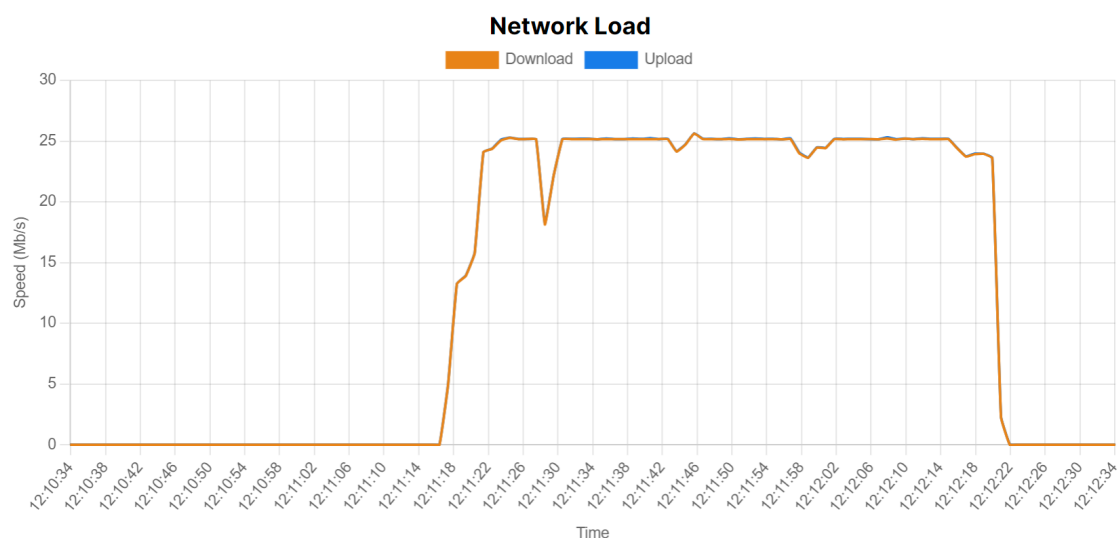
Obr. 6.10: Zatížení síťového rozhraní při testování dopadů HTTP Flood

ACK Flood

Tento útok byl opět proveden pomocí nástroje `Hping3`. S použitím parametru `-ack` nastavíme hlavičky odesílaných TCP paketů na stav `ACK`. I když byl použit přepínač `-flood` nepodařilo se vygenerovat dostatečně velký provoz a ani navzdory snížení rychlosti virtuálního připojení na 24 Mbps se nepodařilo dosáhnout kompletního

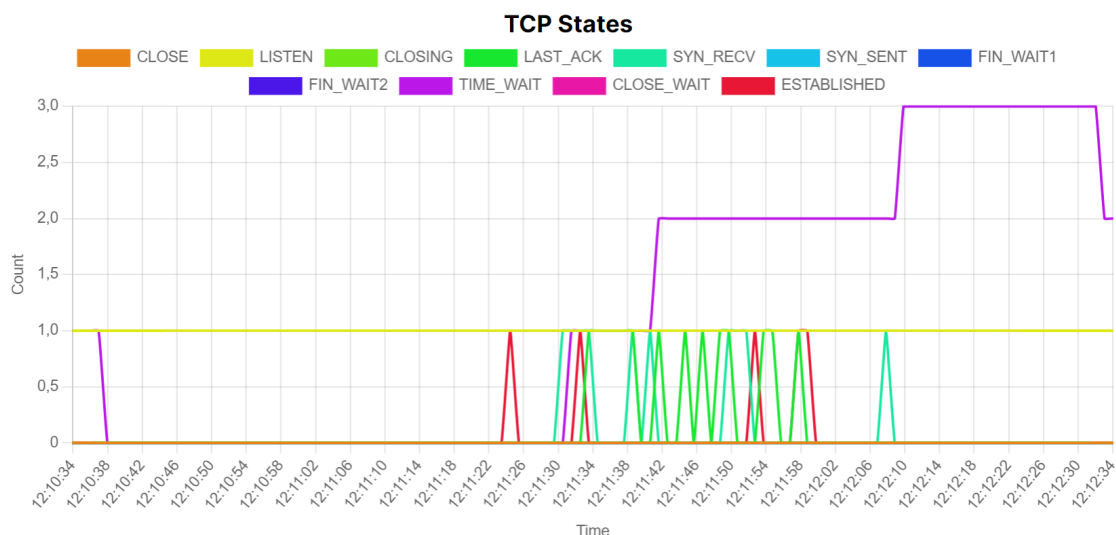
odepření služby. Tento výsledek naznačuje, že ACK flood útok nebyl dostatečně intenzivní, aby plně vyčerpал zdroje serveru nebo způsobil jeho úplnou nedostupnost. Snížení rychlosti připojení mělo za cíl zvýšit účinnost útoku tím, že by server neměl dostatečnou šířku pásma pro zvládání velkého množství paketů, což se však nepodařilo dosáhnout. Ovlivňovat to může také fakt, že mezi útočníkem a obětí nebyl použit běžná router, ale pouze virtuální bridge definovaný v prostředí PROXMOX. Největší naměřený čas odezvy byl 7 600 ms, což je výrazná výjimka ve srovnání s průměrnými časy kolem 1 200 ms. Tento nárůst odezvy mohl být způsoben dočasným zahlcením serveru nebo síťových prostředků.

Zjištěné přenosové rychlosti, kdy upload i download dosahují téměř shodné hodnoty 25,6 Mbps, a počet přijatých a odeslaných paketů 59 332, naznačují, že server musel zpracovat velké množství paketů symetricky. Tento výsledek je charakteristický pro ACK flood útok, který zaplavuje server ACK pakety, na které server odpovídá, což vede k rovnoměrnému zatížení přenosu v obou směrech.



Obr. 6.11: Souměrné zatížení sítě v obou směrech v důsledku ACK Flood

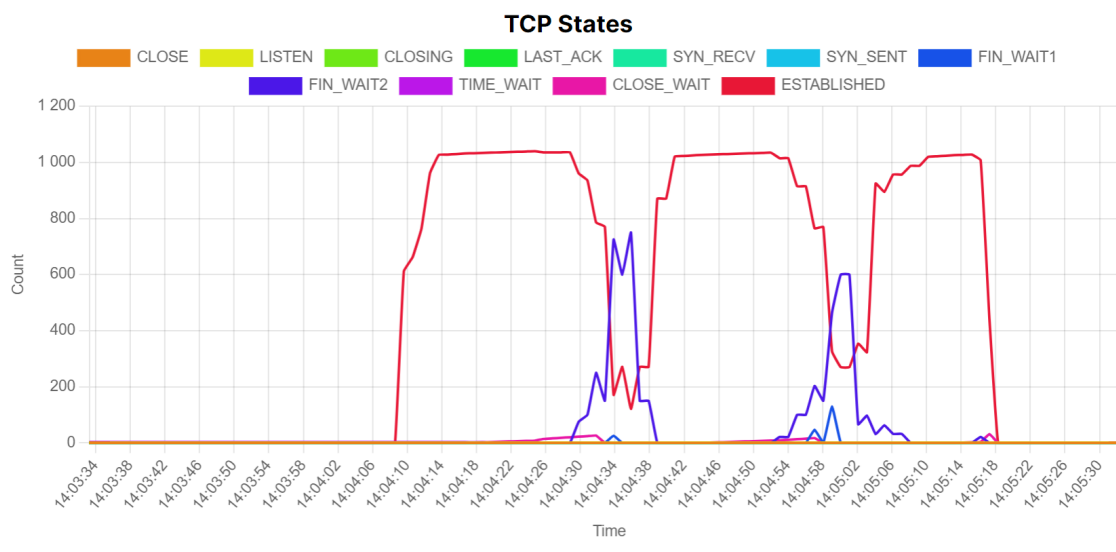
Měření ukazují, že tento útok, při dané intenzitě a konfiguraci, nebyl dostatečný k úplnému odepření služby, ale způsobil zvýšené zatížení serveru a dočasné prodloužení odezvy. Tento typ útoku lze identifikovat na základě specifických znaků, jako je symetrické zatížení síťového rozhraní a vysoký počet přijatých a odeslaných paketů. Skutečnost útoku se ale neprojeví na množství TCP spojení na portu 80, neboť na rozdíl od SYN Flood se útok nesnaží spojení vytvořit, toto tvrzení podkládá graf 6.12.



Obr. 6.12: Stav TCP spojení v průběhu ACK Flood útoku

Slowloris

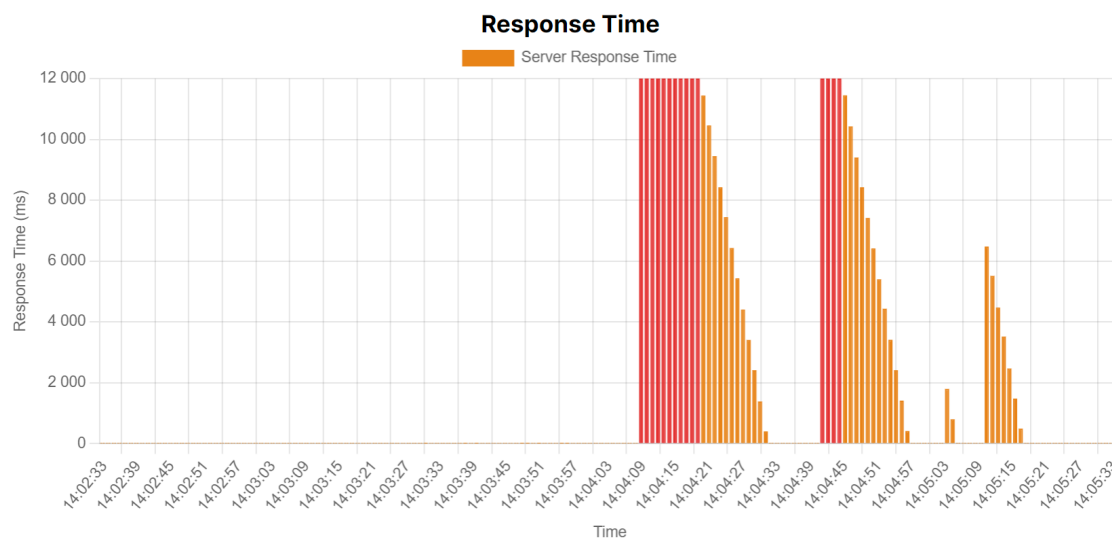
Slowloris je specifický typ DoS útoku, který cílí na vyčerpání zdrojů webového serveru, aniž by zaplavil síť velkým množstvím dat. To bylo také zjištěno při testování, kdy i při relativně malém datovém toku, pohybujícím se v jednotkách Mbps, bylo otevřeno až 1000 spojení. Tyto spojení se postupně po 20 sekundách zavírala a následně byla navazována nová. Celkem byly pozorovány tři vlny nových spojení, které vždy dočasně uvedly Apache server do stavu odepření služby. Podrobnější pohled na průběh nabízí graf 6.13.



Obr. 6.13: Stavy TCP spojení při útoku Slowloris

Toto chování je typické pro Slowloris útok, který využívá techniky částečných HTTP požadavků k udržení spojení otevřených co nejdéle. Server alokuje prostředky pro každé neukončené spojení, což vede k vyčerpání dostupných slotů pro nové požadavky.

Měření ukázala, že nárůst zatížení je pozorovatelný jak na CPU, tak na RAM. Zatížení CPU je způsobeno zpracováváním velkého počtu otevřených spojení a neustálou správou stavu těchto spojení. RAM je zatížena kvůli potřebě udržovat informace o každém neukončeném spojení.



Obr. 6.14: Změřená odezva serveru v průběhu útoku Slowloris

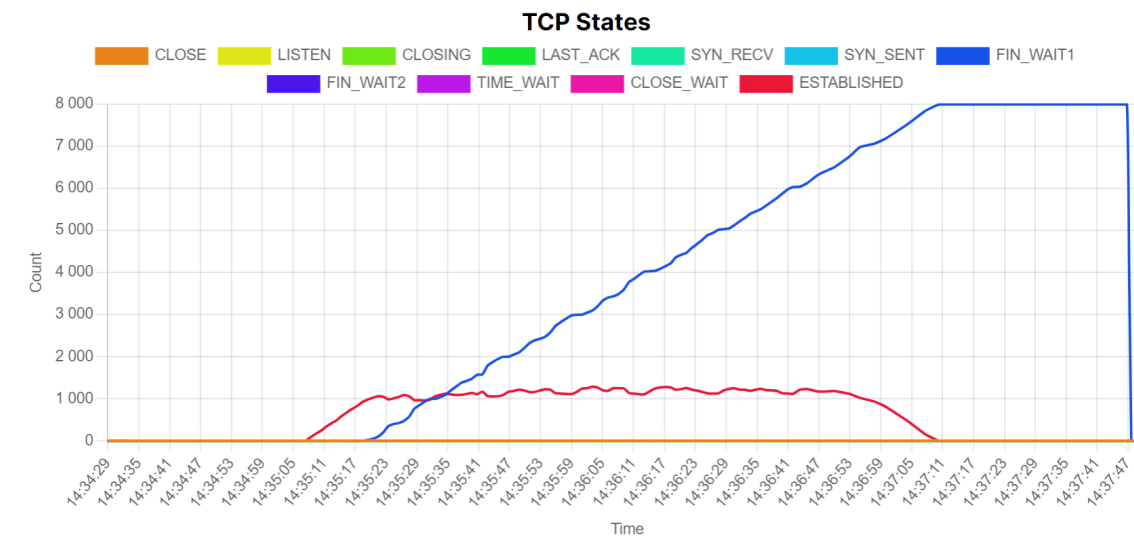
Během útoku bylo pozorováno, že navzdory malému datovému toku dokázal Slowloris útok efektivně uvést Apache server do stavu odepření služby. Tři vlny nových spojení, které následovaly po každém cyklu uzavření starých spojení, vedly k opakovanému zatížení serveru a dočasné nedostupnosti služeb. Server měl v době útoku 1 000 workerů a parametr `KeepAliveTimeout` nastaven na 30 s.

Slow READ

Pro generování Slow Read útoku byl použit nástroj `pyslowdos.py`, který efektivně simuloval velké množství pomalých klientů, kteří udržovali dlouhá otevřená spojení a pomalu četli data od serveru. Stejně jako v předchozím případě měl server k dispozici 1 000 workerů a parametr `KeepAliveTimeout` nastaven na 30 s.

Při útoku byla na síťovém rozhraní změřena relativně nízká komunikace, která se opět pohybovala v jednotkách Mbps. Bylo pozorováno, že upload byl o něco vyšší než download, což naznačuje asymetrický přenos dat typický pro Slow Read útoky, ovšem počet přenesených paketů byl stejný pro upload i download. Tento

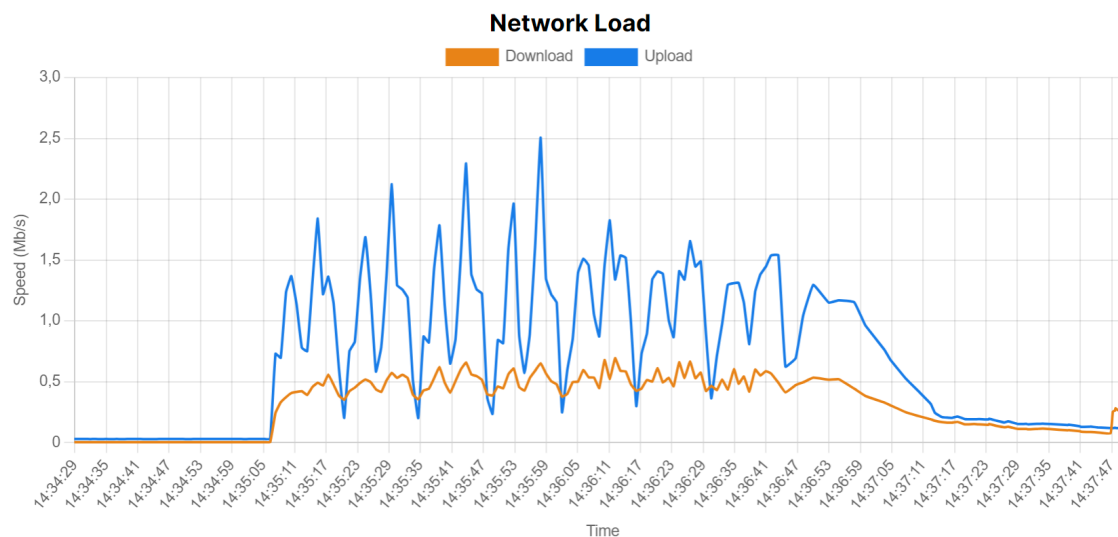
typ asymetrie je způsoben tím, že útočník generuje malé množství požadavků, ale nutí server odesílat větší odpovědi, které jsou následně čteny velmi pomalu. Tím se vytváří nerovnoměrné zatížení přenosu dat mezi klientem a serverem. Tuto asymetrii můžeme pozorovat na grafu 6.16.



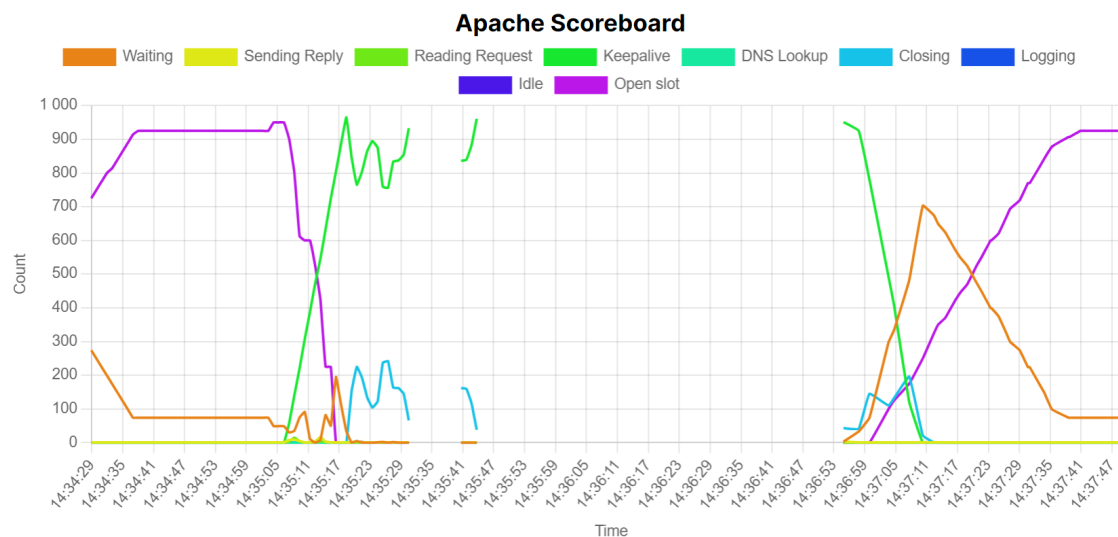
Obr. 6.15: Stavy TCP spojení v průběhu útoku Slow READ

Výrazný nárůst zatížení procesoru a operační paměti je opět způsoben potřebou serveru udržovat a spravovat velké množství otevřených spojení, nutností alokovat pro každé z nich paměť a udržovat stav těchto spojení po dlouhou dobu. Postupně bylo zjištěno přibližně 1200 aktivních spojení ve stavu **ESTABLISHED** a jak útok pokračoval, tato spojení postupně přecházela do stavu **FIN_WAIT1**, kde se nasčítalo až 8000 spojení do ukončení útoku. Tento stav naznačuje, že server čekal na dokončení uzavření jednotlivých spojení, což dále zatěžovalo jeho zdroje.

Statistika Apache serveru během útoku ukazovala přibližně 4,6 požadavků za sekundu a 6044 bajtů na jeden požadavek. Tyto hodnoty ukazují na relativně nízký počet požadavků, ale větší objem dat na požadavek. I přesto, že se jedná o Slow READ útok, server stále musí zpracovat celé odpovědi na každé z těchto požadavků, což vede k většímu objemu dat na jeden požadavek. Statistiky webového serveru dále nabízí pohled na scoreboard, kde je patrné jak postupně všechny workeri přešli ze stavu **idle** „“ do stavu **keepalive** „K“, kde čekaly na další požadavky od klientů. Jak se postupně více workerů přesouvalo do stavu **keepalive**, došlo k vyčerpání dostupných workerů, což nakonec vedlo k výpadku serveru a odepření služby. Tento výpadek je též pozorovatelný na samotných statistikách Apache na grafu 6.17.



Obr. 6.16: Množství procházejících dat síťovým rozhraním oběti při útoku Slow READ



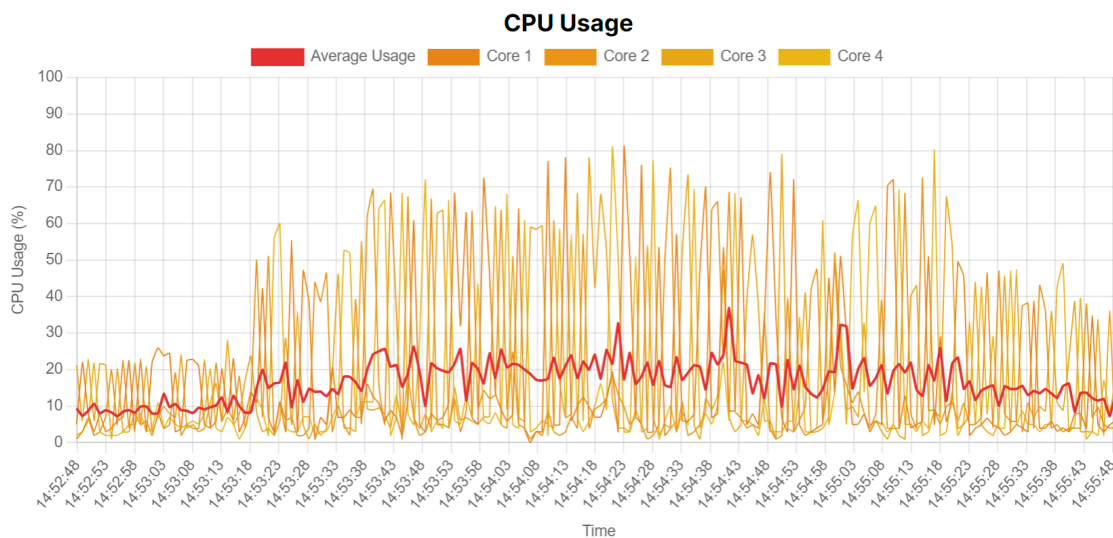
Obr. 6.17: Apache scoreboard při útoku Slow READ

Slow POST

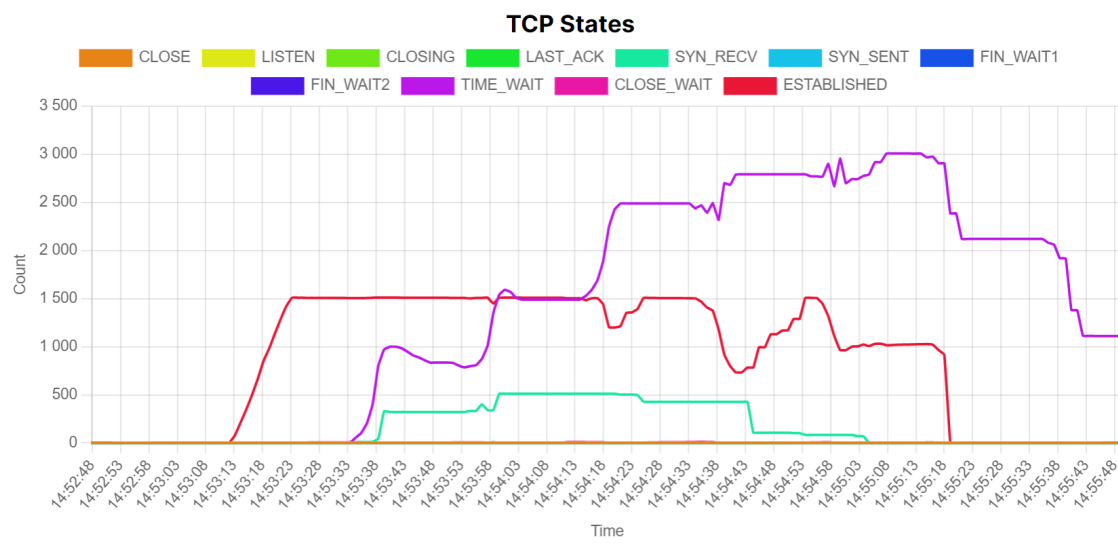
K tomuto útoku byl využit nástroj `slowhttptest` ve verzi 1.8.2. Při testování pak byly pozorovány podobné příznaky jako u předchozích útoků, avšak zde nebyl patrný rozdíl mezi uploadem a downloadem. Tento útok generuje rovnoměrný přenos dat v obou směrech, což naznačuje, že server musí přijímat a zpracovávat data od útočníka kontinuálně, zatímco útočník pomalu odesílá POST data.

Zatížení CPU bylo při Slow POST útoku nižší než při Slow Read útoku. Tento rozdíl může být způsoben tím, že server primárně čeká na dokončení přenosu dat v těle POST požadavku a není tak intenzivně zatěžován správou stavu spojení jako při Slow Read útoku. Možná příčina tohoto jevu je také to, že tento útok byl generován s mnohem menším počtem spojení. Na úrovni TCP bylo pozorováno postupné vytváření spojení ve stavu `ESTABLISHED`, které dosáhlo až 1500 spojení. Po dosažení této úrovně se růst zastavil a začaly přibývat spojení ve stavu `TIME_WAIT`, což postupně narostlo až na 3500. Stav `SYN_RECV` se pohyboval kolem 400, což ukazuje na probíhající pokusy o navázání nových spojení. Kompletní přehled stavů TCP na portu 80 nabízí graf 6.19.

Při testování se podařilo zcela zaneprázdnit Apache server, což se projevilo vysokým počtem spojení a nedostupností serveru do doby 12 s. Server byl zaneprázdněn správou neukončených spojení a čekáním na dokončení POST požadavků, což vedlo k výraznému zhoršení jeho výkonu a odepření dostupnosti služeb.



Obr. 6.18: Zátěž CPU během útoku Slow POST



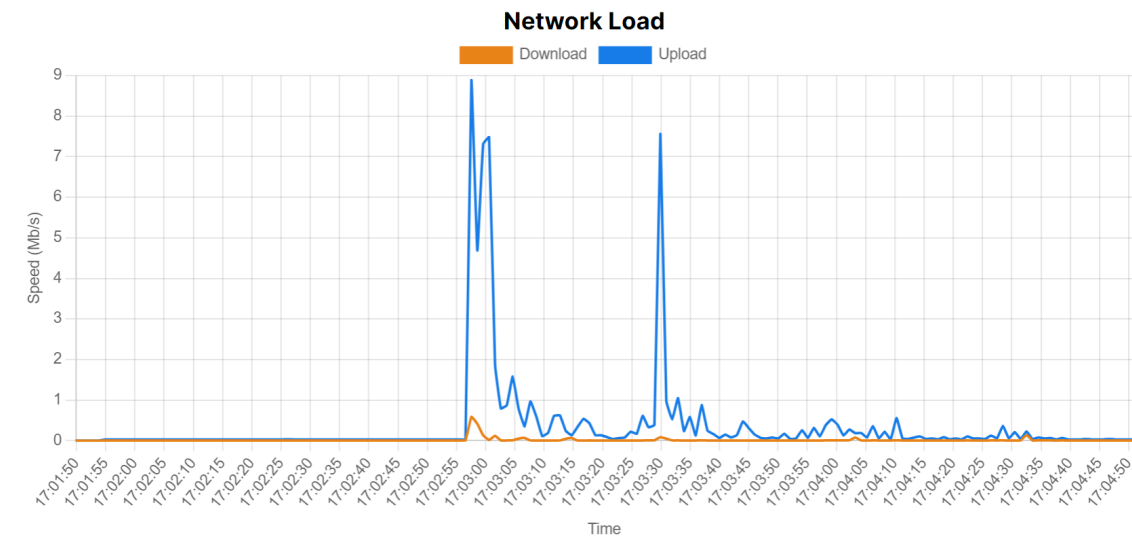
Obr. 6.19: Jednotlivé stavy TCP spojení při útoku Slow POST

Slow DROP

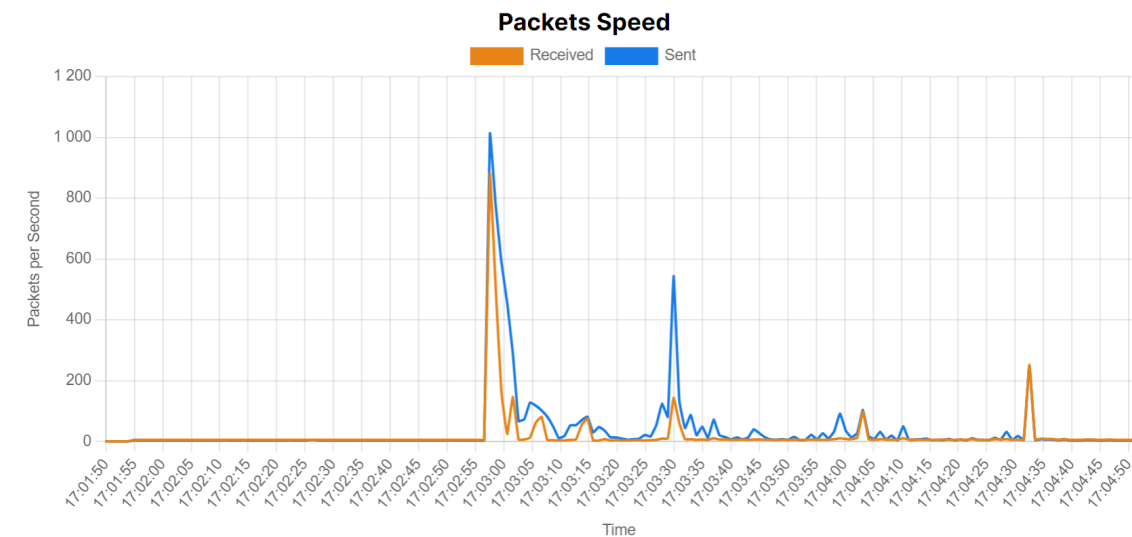
Pro tento útok byl znovu použit nástroj `pyslowdos.py`, který nabízí možnost zvolení jak často má být příchozí paket zahozen, tedy `DROP RATE`, což simuluje poruchové spojení s velkým množstvím výpadků. V tomto konkrétním měření byla použita hodnota 0,6. Z dat získaných při útoku Slow Drop můžeme vyčíslit, že zatížení CPU bylo menší než u předchozích útoků a na RAM paměť neměl útok téměř žádný vliv. To naznačuje, že tento útok méně intenzivně zatěžuje výpočetní zdroje serveru a nevyžaduje výrazné alokace paměti.

Z měření síťového rozhraní zachyceném na grafu 6.20, vystupují špičky uploadu až na 8 Mbps, zatímco download zůstal okolo 0,5 Mbps. Tento rozdíl indikuje, že je útok schopen za pomoci malého množství odeslaných dat, donutit server vygenerovat velký datový tok. Naměřená data paketů na grafu 6.21 ale naznačují, že počet příchozích a odchozích paketů je podobný, což by nebylo typické pro Slow Drop útok, pokud by server musel neustále znovu odesílat nové pakety, očekávali bychom větší asymetrii v počtu paketů. Tato nesrovnalost může být následkem nedostatečné intenzity útoku, chybným nastavením testu, nebo příliš malé velikosti načítané stránky.

Graf ukazující odezvu serveru ovšem jasně identifikuje že 30 vteřin byl server nedostupný, což se shoduje s dobou, kdy byly aktivní TCP spojení než došlo k jejich změně na stav `FIN_WAIT`.



Obr. 6.20: Zatížení síťového rozhraní při útoku Slow DROP

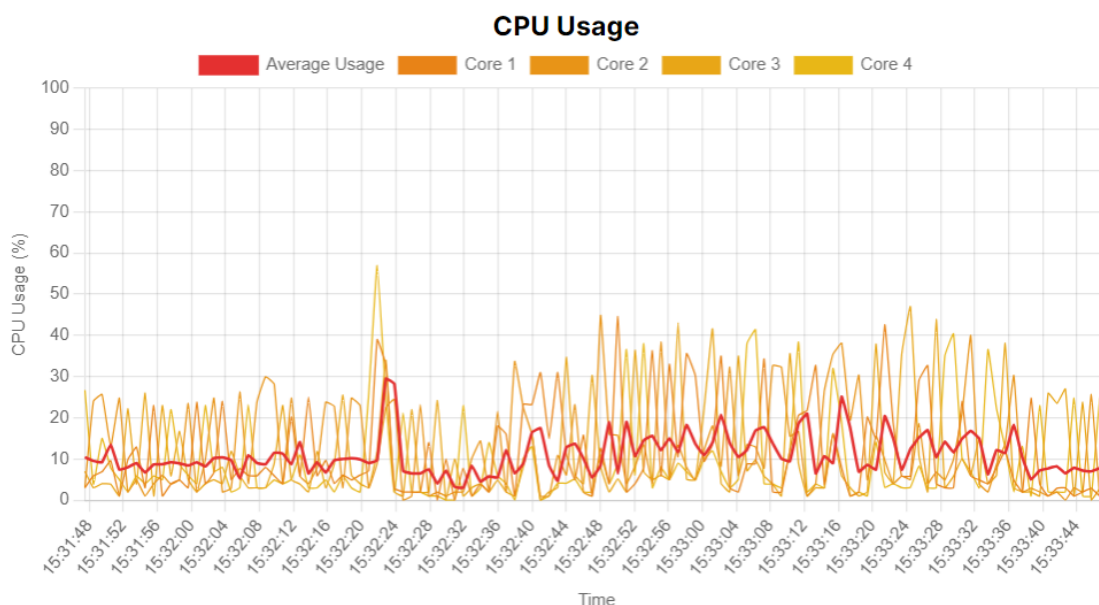


Obr. 6.21: Počet paketů za sekundu během útoku Slow DROP

Slow NEXT

I při tomto útoku byl znovu použit nástroj `pyslowdos.py`. Při tomto testu se nepodařilo dosáhnout stavu kompletního odepření služby, ale došlo alespoň k výraznému nárůstu doby odezvy, průměrně na 2 800 ms. Jelikož se často ale dostala odezva serveru nad 3 vteřiny, jsou na grafu 6.24 viditelné výpadky. Celkem tak bylo v několika chvílích zaměstnáno všech 200 workerů. Zatížení CPU a RAM bylo velmi malé, což naznačuje, že útok nevyžaduje intenzivní využití výpočetních zdrojů, neboť server většinu času pouze čeká na vypršení `KeepAliveTimeout` aby mohl spojení uzavřít. Tento parametr byl nastaven na 30 s.

Prvotní navázání spojení je i u tohoto útoku nárazové a došlo při něm k navázání přibližně 252 spojení přičemž můžeme na grafu vidět, že jejich udržování bylo poměrně stabilní po celou minutu probíhajícího útoku. Z dat získaných ze síťového rozhraní lze vyčíst, že útok obnovoval spojení každých 5 vteřin a serverová odpověď byla výrazně větší, než žádost. To může být mimo jiné dáno obsahem načítané stránky.

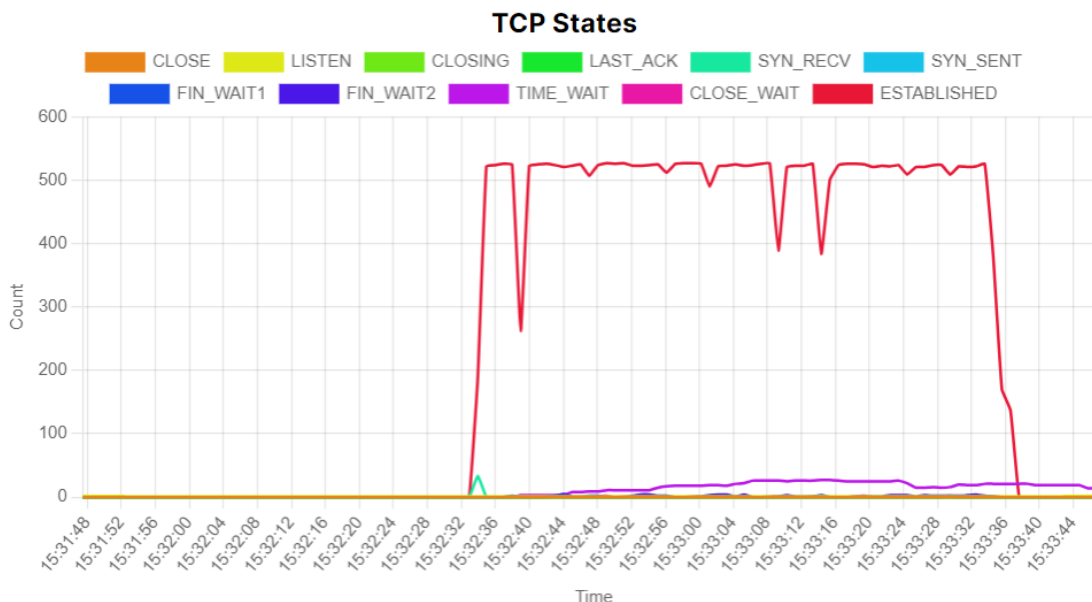


Obr. 6.22: Nárůst zatížení procesoru při útoku Slow NEXT

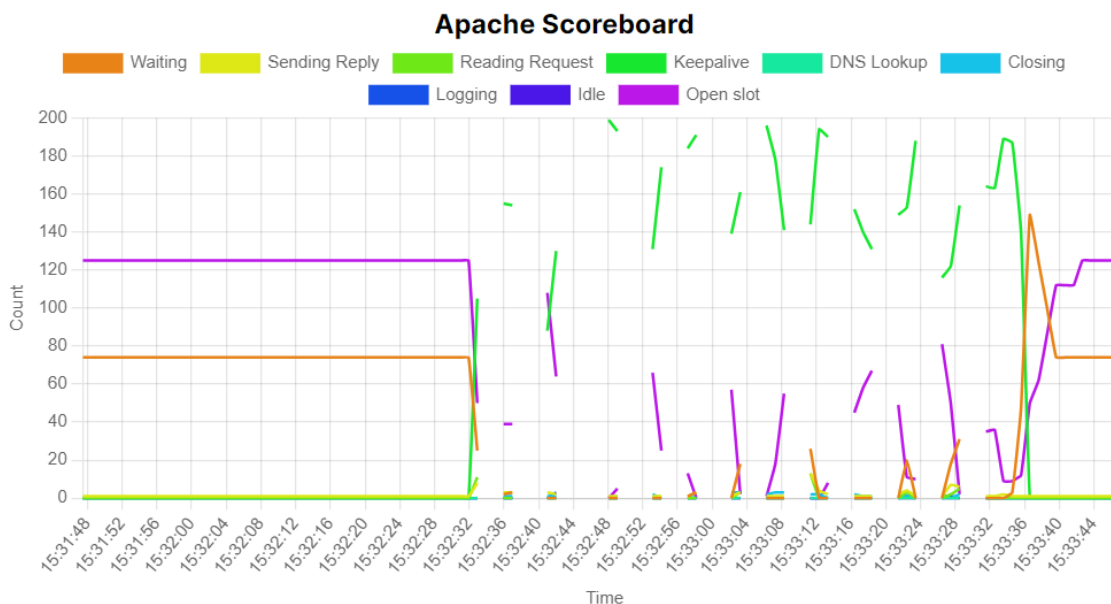
Shrnutí

Ze získaných dat je patrné, že reakce serveru na jednotlivé typy útoků se výrazně liší, což znemožňuje stanovit univerzálně platné konstanty jednotlivých parametrů, při jejichž překročení by bylo možné jednoznačně určit, že se jedná o útok. Například

každý útok má specifické charakteristiky, které ovlivňují různé části systému. Některé útoky, jako SYN Flood, vedou k výraznému zatížení CPU a zvýšenému počtu neuzavřených TCP spojení. Na druhé straně útoky jako Slowloris způsobují nárůst počtu otevřených spojení, která zatěžují paměťové a procesorové zdroje, i když jsou přenosová data relativně malá.



Obr. 6.23: Vliv Slow NEXT útoku na aktivní TCP spojení



Obr. 6.24: Apache scoreboard při útoku Slow NEXT

6.5 Metoda sledování TCP spojení

Než budeme vyvozovat metody ze zjištěných dat je třeba vzít v úvahu, že testovací server nebyl zatížen běžným provozem, což znamená, že výsledná data mohou být zkreslená. V reálném prostředí by se DoS útoky mísily s legitimním provozem, což by ztížilo jejich detekci. Musíme brát v potaz, že některé „signatury“, které útoky vykazují, může vytvořit i běžný provoz. Z měření je také patrné, že pokud webové servery pracují na hranici svých možností, jsou pro DoS snadným cílem a samotný útok pak bude méně patrný.

Útoky je tedy vhodné detekovat odchylkou od normálu. Nejjednodušší detekci můžeme provést analýzou TCP spojení na portu webového serveru. Tato data máme ve webové aplikaci k dispozici v grafu **Sockets on Port**. Ve 2 po sobě jdoucích záznamech můžeme spočítat absolutní hodnotu rozdílu spojení a na jejím základě pak sestavit graf. Pokud je běžná změna v jednotkách spojení a najednou máme vrchol ve stovkách, je jasné, že se nejedná o náhodu kdy se stovky uživatelů během jedné vteřiny pokouší přistoupit na náš web.

Matematicky můžeme tento okamžik definovat za pomoci absolutní hodnoty změny počtu spojení.

$$\Delta S_t = |S_t - S_{t-1}|$$

- ΔS_t je absolutní hodnota změny počtu spojení
- S_t je počet aktuální počet spojení
- S_{t-1} je předchozí počet spojení

V tuto chvíli můžeme postupně projít naměřené hodnoty a vypočítat jejich absolutní rozdíly. Příklad jak můžou výsledky těchto jednotlivých výpočtů vypadat znázorňuje graf 6.25.

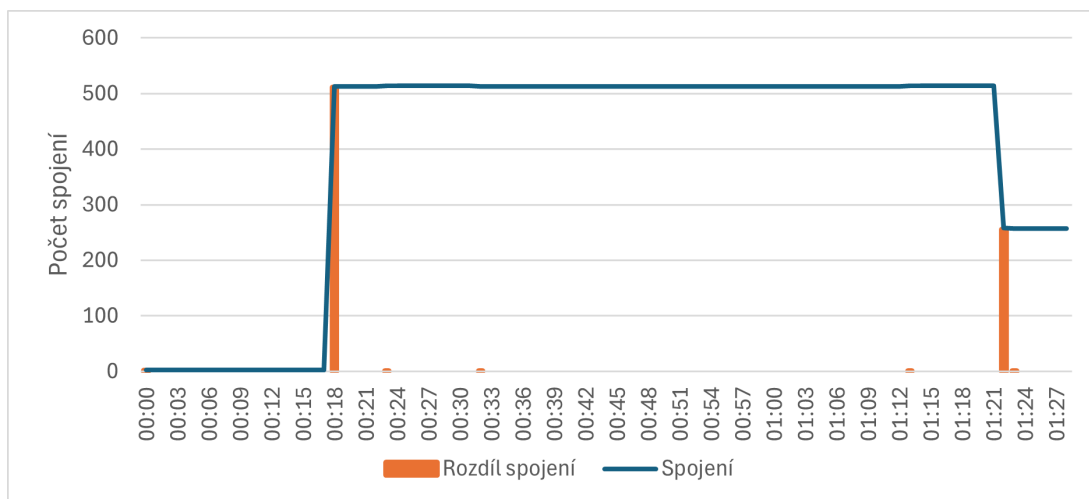
Pro samotnou detekci je potřeba si nejprve nadefinovat hodnotu T_h , která bude sloužit jako práh detekce.

$$T_h = k \cdot S_{\max}$$

- T_h definovaný práh detekce
- $S_{\max}$ je maximální počet spojení serveru
- k je koeficient citlivosti detekce (například 0.1 znamená 10 % maximálního počtu spojení)

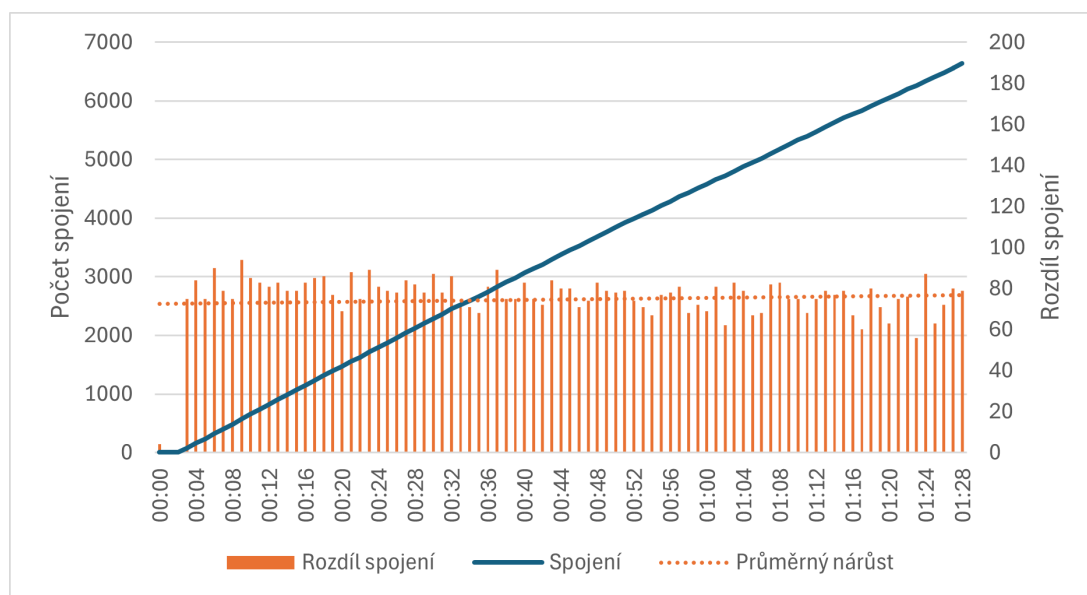
Tímto způsobem máme určený práh detekce a už jej stačí porovnat s předchozími výsledky. Pokud pak platí vztah $\Delta S_t > T_h$, můžeme prohlásit útok za detekovaný pomocí prudkého nárůstu spojení.

Pokud je tento nárůst pozvolný a není pravidelný, může se jednat o situaci vzniklou přirozeně, například kvůli zahájení předprodeje očekávaného výrobku nebo



Obr. 6.25: Zobrazení závislosti počtu aktivních spojení a jejich změně na čase od začátku testu útoku SYN flood

zveřejnění důležitého obsahu. V opačném případě kdy na grafu připojení vidíme schody a změna spojení za vteřinu je konstantní, můžeme znovu identifikovat DoS. Tento případ je na grafu 6.26. Přidaná spojnice trendu nám ukazuje průměrný počet nových spojení za sekundu. Z logického hlediska je nereálné, aby se uživatelé připojovali v takto přesné šabloně.



Obr. 6.26: Lineární nárůst spojení TCP při testu útoku Slow READ

Pokud chceme tuto skutečnost vyjádřit matematicky, musíme pracovat s průměrem změn ΔS_t několika posledních měření n a oproti němu pak jednotlivé měření porovnat. Jelikož nárůst spojení může být ovlivněn výkyvy v síti je třeba přidat kon-

stantu tolerance výchyly ϵ . Pokud není tato výchyly v toleranci, může se jednat o výkyv v měření a proto je třeba přidat další parametr l , který určuje minimální procento měření v toleranci výchyly ϵ . Začneme spočítáním samotného průměru ΔS_{avg} .

$$\Delta S_{\text{avg}} = \frac{1}{n} \sum_{i=1}^n |S_{t-i} - S_{t-i-1}|$$

- S_t je počet TCP spojení v čase t
- S_{t-i} je počet TCP spojení v čase $t - i$
- n je počet časových intervalů, přes které se průměrný nárůst počítá.
- ΔS_{avg} je průměrná změna počtu spojení

A nyní můžeme ověřit jednotlivé tolerance.

$$\left(\frac{1}{n} \sum_{i=1}^n \left(\left| \frac{\Delta S_{\text{avg}} - \Delta S_{t-i}}{\Delta S_{\text{avg}}} \right| \leq \epsilon \right) \right) \geq l$$

- ΔS_{avg} je průměrná změna počtu spojení za posledních n časových intervalů.
- ΔS_{t-i} je rozdíl spojení $\Delta S_{t-i} = |S_{t-i} - S_{t-i-1}|$
- ϵ je toleranční hodnota pro relativní rozdíl, která definuje, jak blízko musí být změna počtu spojení k ΔS_{avg}
- l je minimální procento intervalů, které musí splňovat podmínku, aby byl detekován DoS útok

Pokud je touto operací detekován DoS, můžeme prohlásit, že byl detekován na základě konstantního nárůstu připojení.

Sledování nárůstu aktivity TCP pomocí grafu si můžeme ulehčit například pomocí programu Microsoft Excel, kam můžeme importovat CSV data z webové aplikace. Jednoduchou funkcí `=ABS(<aktuální hodnota> - <předchozí hodnota>)` na sloupec hodnot `socketsOnPortArray` získáme potřebné data ΔS_t , které můžeme následně zobrazit v grafu, jako například ukázka 6.25 a data vizuálně porovnat.

Při detekci se ale nemusí jednat pouze o sledování počtu spojení na portu 80, ale také o analýzu jednotlivých stavů těchto spojení. Stav `SYN_RECV` například označuje spojení, které čeká na dokončení TCP handshake procesu. Velké množství spojení ve stavu v tomto stavu může indikovat SYN Flood útok. Spojení ve stavu `ESTABLISHED` zase znamenají, že TCP handshake byl úspěšně dokončen a komunikace mezi klientem a serverem probíhá. Analýza nárůstu spojení v tomto stavu může pomoci identifikovat Slowloris nebo Slow Read útoky, kde útočník udržuje dlouhá spojení otevřená a pomalu odesílá data, což vyčerpává zdroje serveru. Soubor, který je prostřednictvím přiložené aplikace možno stáhnout poskytuje tato data také v separované podobě, což umožňuje jejich podrobnější analýzu.

Při této detekční metodě si také můžeme pomoci ukazatelem doby odezvy serveru. Sledování této hodnoty je při identifikaci DoS útoků vhodné, protože výrazné prodloužení odezvy může být indikátorem přetížení serveru způsobeného útokem a napovídá nám na jaké období se máme soustředit.

6.5.1 Testování metody

Navržená metoda pro detekci DoS útoků na základě sledování změn v počtu TCP spojení má své omezení. Přestože je účinná při identifikaci určitých typů útoků, není schopna detekovat všechny útoky, například ICMP Flood, který je proti této metodě zcela imunní. Abych ověřil funkčnost metody, nejenže jsem prováděl testování sepsanými funkcemi, ale zároveň jsem vytvářel grafy, které reprezentovaly rozdíl v počtu spojení a hledal v nich specifické signatury popsané v této metodě. Tato vizuální reprezentace poskytuje jasnější a názornější pohled na detekci útoků a je schopna okamžitě vyřadit nedetekovatelné útoky.

Následující tabulka 6.1 ukazuje, které útoky tato metoda odhalí. Metoda je použitelná pouze pro útoky pracující s TCP spojením. Ovšem i této metodě se lze vyhnout, například randomizováním doby, během které se vytvoří další spojení, a počtu těchto spojení. Výsledkem jsou odchylky podobné běžnému provozu, který není konstantní, což může způsobit, že útok nebude detekován.

Tab. 6.1: Testování metody sledováním TCP spojení

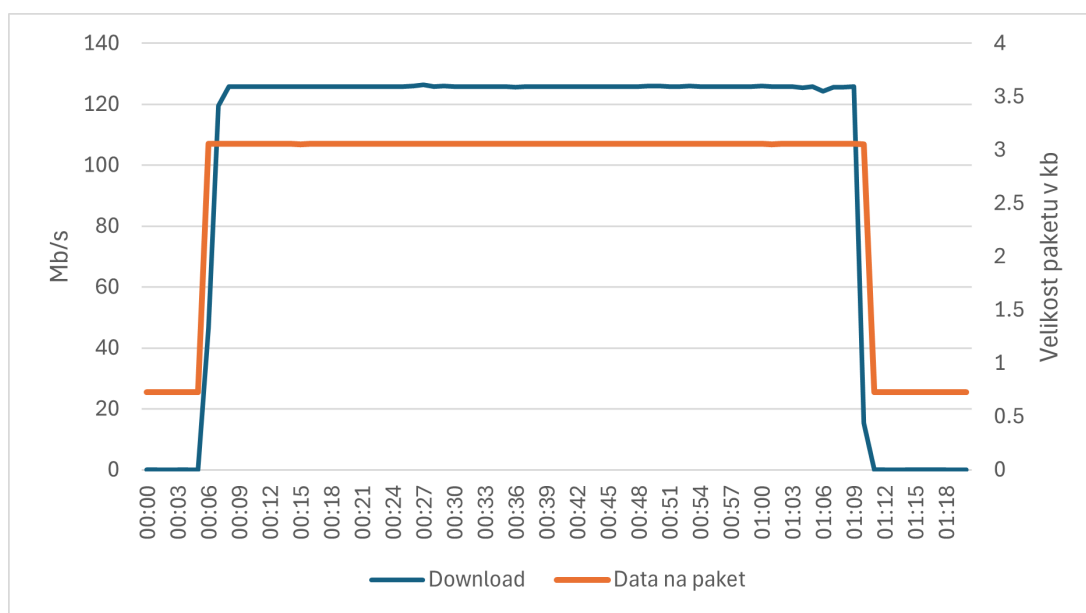
Útok	Detekovaná signatura
ICMP Flood	Nedetekován
UDP Flood	Nedetekován
SYN Flood	Pravidelný nárůst spojení
HTTP Flood	Nedetekován
ACK Flood	Nedetekován
Slowloris	Prudký nárůst spojení TCP
Slow READ	Pravidelný nárůst spojení
Slow POST	Pravidelný nárůst spojení
Slow DROP	Prudký nárůst spojení TCP
Slow NEXT	Prudký nárůst spojení TCP

Přestože metoda není dokonalá a má svá omezení, poskytuje užitečný nástroj pro detekci specifických typů DoS útoků. Je důležité si uvědomit, že žádná metoda není univerzálně účinná proti všem typům útoků a že kombinace různých metod a technik poskytne nejlepší ochranu.

6.6 Metoda sledování poměru paketů a množství dat

Identifikace záplavových útoků může být také prováděna sledováním poměru počtu paketů k objemu přenesených dat. Záplavové útoky se často vyznačují vysokým počtem malých paketů, které mohou být identifikovány tímto poměrem. Nicméně je nezbytné sledovat i další parametry, protože monitorovací skripty pro sledování stavu serveru mohou rovněž generovat malé pakety, často dokonce menší než ty, které jsou generovány při útoku ICMP Flood. Rozdíl mezi legitimním provozem a útokem spočívá v tom, že útoky mají tendenci zaplnit celou šířku komunikačního pásma.

Cílem této metody je tedy identifikovat okamžiky, kdy síťovým rozhraním prochází vysoké množství relativně malých paketů. Tuto analýzu lze provádět pomocí nástrojů jako je Microsoft Excel, kde lze vytvořit grafy zobrazující velikost paketů v čase. Tento přístup může odhalit skryté vzorce, které by jinak nebyly viditelné. Například během výkyvů v přenosové síti by mohlo dojít k téměř maximálnímu využití šířky pásma, ale pokud není počet přenesených paketů a rychlost přenosu konstantní, mohlo by dojít k přehlédnutí skutečného útoku. Analýzou poměru mezi počtem paketů a objemem dat, můžeme identifikovat konstantní velikost paketů, což je indikátorem záplavového útoku. Ukázkou takové signatury nabízí graf 6.27.



Obr. 6.27: Zobrazení konstantní velikosti paketů při ICMP Flood útoku

Matematický zápis této metody je možný a bude obdobný předchozímu příkladu. Je však třeba zdůraznit, že vyhodnocení touto metodou je vhodné pouze v případech, kdy je zatížení síťového rozhraní vysoké, protéká velké množství malých paketů a existuje podezření na probíhající DoS útok. Tato metoda totiž neuvažuje maximální velikost balíčku, ale porovnává pouze to, zda je přenos konstantní. Je tedy

vhodné metodě poskytovat pouze data, která odpovídají těmto podmínkám a ty následně analyzovat. Nejprve je nutné ale pro všechny změřené hodnoty n určit velikost paketů V_t na základě které můžeme poté pokračovat.

$$V_t = \frac{D_t}{P_t}$$

- V_t je velikost paketů v čase t
- P_t je počet paketů v čase t
- D_t je objem dat v čase t

Nyní je potřeba zjistit, zda je tato velikost konstantní, nebo proměnná. K tomu využijeme funkci z předchozí metody. Začneme opět spočítáním průměrné velikosti paketu za posledních n intervalů.

$$V_{\text{avg}} = \frac{1}{n} \sum_{i=1}^n V_{t-i}$$

- V_{avg} je průměrná velikost paketu
- V_{t-i} je velikost paketu v čase $t - i$

Následně můžeme porovnat jednotlivé velikosti V_t s průměrem V_{avg} a sledovat případné odchylky.

$$\left(\frac{1}{n} \sum_{i=1}^n \left(\left| \frac{V_{\text{avg}} - V_{t-i}}{V_{\text{avg}}} \right| \leq \epsilon \right) \right) \geq l$$

- n je počet porovnávaných intervalů
- V_{avg} je průměrná velikost paketu
- V_{t-i} je velikost paketu v čase $t - i$
- ϵ je toleranční hodnota pro relativní rozdíl
- l je minimální procento intervalů, které musí splňovat podmínku, aby byl detekován DoS útok

Pokud tato funkce platí, můžeme prohlásit detekování útoku DoS na základě velkého a konstantního přenosu malých paketů.

Legitimní provoz, jako je stahování souborů, může rovněž způsobit konstantní velikost paketů, ale zde bude velikost paketů obvykle větší než při záplavových útocích. I když může dojít k plnému využití přenosové kapacity, velikost paketů při stahování souborů bude vyšší, protože tento typ provozu typicky přenáší větší bloky dat.

Obecně platí, že při sledování množství dat procházejících síťovým rozhraním, je důležité v případě plného využití šířky pásma analyzovat také velikost paketů. Pokud zaznamenané velký tok malých paketů, je pravděpodobné, že se jedná o DoS útok,

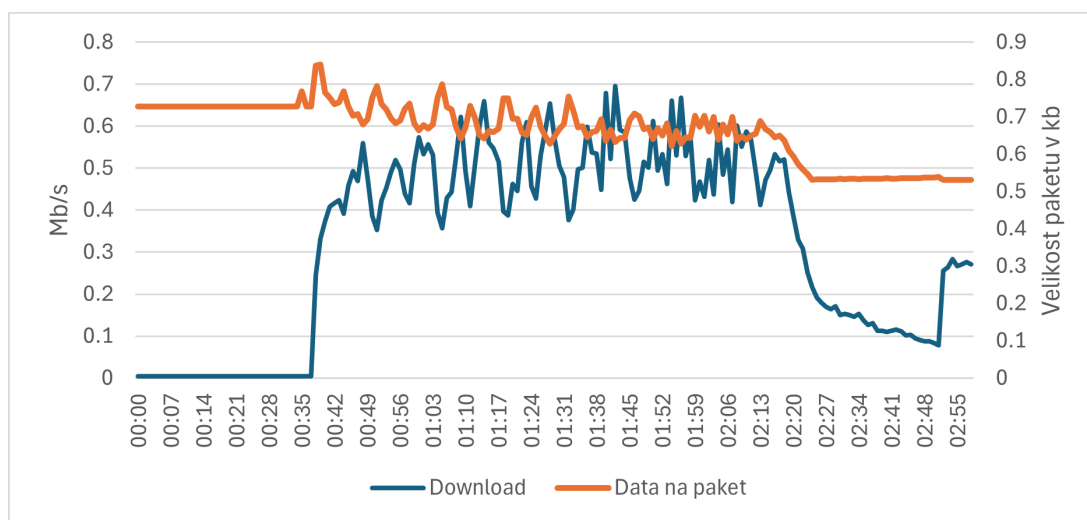
protože tento vzorec není typický pro přirozený provoz. Konstantní či pravidelně se opakující vzorce přenosu nejsou generovány lidskou aktivitou, ale automatizovanými systémy. Ovšem, ne všechny tyto vzorce jsou nutně škodlivé. Legitimní provoz může zahrnovat například periodické stahování aktualizací nebo synchronizaci dat, což by nemělo být mylně identifikováno jako DoS útok.

Důležitou součástí této metody je tedy nejen sledování absolutních hodnot, ale také porozumění kontextu, ve kterém tyto hodnoty vznikají. To zahrnuje nejen technické aspekty, jako je velikost a počet paketů, ale také znalost běžného provozu v daném prostředí, což umožňuje rozlišit mezi normálními aktivitami a potenciálními útoky.

Zohlednění všech těchto faktorů umožňuje jednoznačnou identifikaci útoků na základě podrobných vzorců chování síťového provozu, a zároveň minimalizuje riziko falešných poplachů při běžných činnostech.

6.6.1 Testování metody

Samotné testování této metody probíhalo pomocí analýzy poměru z naměřených hodnot jednotlivých útoků a jejich následného zobrazení v grafu. Přestože tato metoda nemusí odhalit tak širokou škálu útoků jako předchozí metoda, její využitelnost zůstává vysoká, protože se zaměřuje na specifické typy útoků, které mohou být jinými technikami přehlédnuty. Příkladem útoku, který tato metoda neidentifikuje je Slow READ, jehož měření ukazuje graf 6.28.



Obr. 6.28: Útok Slow READ, který není metodou sledování velikosti paketů detekován

Během testování jsem spočítal poměry mezi objemem dat a počtem paketů pro každý z testovaných útoků. Tyto poměry byly poté zobrazeny v grafické podobě, což

umožnilo snadno vizualizovat vzorce chování různých typů útoků. Výsledky testování ukázaly, že tato metoda je zvláště efektivní při identifikaci záplavových útoků, které se vyznačují velkým počtem malých paketů.

Výsledky testování této metody jsou shrnuty v následující tabulce 6.2, která ukazuje efektivitu detekce jednotlivých typů útoků pomocí této techniky.

Tab. 6.2: Testování metody velikosti paketu

Útok	Detekovaná signatura
ICMP Flood	Plné zatížení přenosového pásma malými pakety
UDP Flood	Plné zatížení přenosového pásma malými pakety
SYN Flood	Nedetekován
HTTP Flood	Nedetekován
ACK Flood	Nedetekován
Slowloris	Nedetekován
Slow READ	Nedetekován
Slow POST	Nedetekován
Slow DROP	Nedetekován
Slow NEXT	Nedetekován

Důležitým zjištěním je také skutečnost, že metoda je citlivá na charakteristiky legitimního provozu, což znamená, že je nutné pečlivě nastavit prahové hodnoty a parametry pro detekci, aby se minimalizovalo riziko falešných poplachů. V praxi by měla být tato metoda aktivována jen v případech, vysokého zatížení serverového rozhraní a velkého množství paketů.

Celkově lze ale říci, že metoda sledující změnu velikosti paketů a množství dat představuje účinný nástroj pro identifikaci těch typů DoS útoků, které využívají malé pakety k vyčerpání síťových zdrojů. Při správné implementaci a kalibraci nabízí tato metoda cenný přínos k celkové strategii ochrany proti DoS útokům.

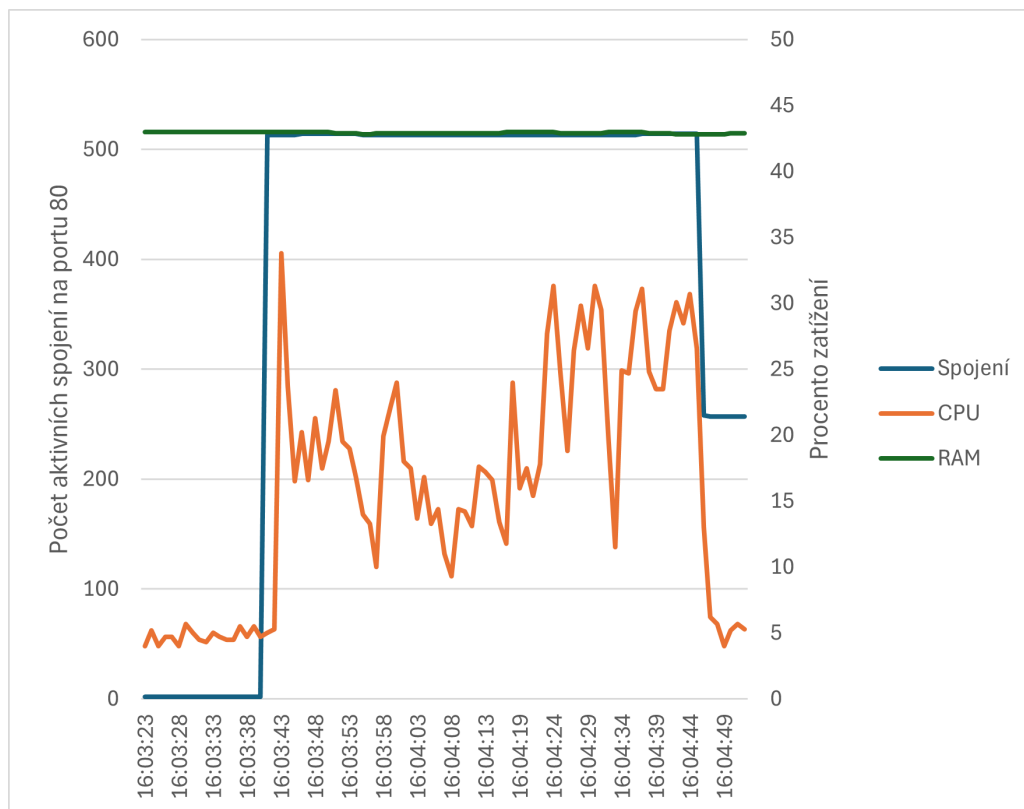
6.7 Závislost mezi hardwarovými a softwarovými zdroji

Závislosti mezi hardwarovými a softwarovými zdroji při zatížení DoS útokem jsou důležité pro pochopení, jak různé typy útoků ovlivňují výkon serveru. Na ukázkou jsem vybral jeden záplavový (SYN flood) a jeden logický útok (Slowloris).

Data o vlivu těchto útoků byla shromážděna pomocí webové aplikace a dále exportována do formátu CSV a zobrazena pomocí grafu v aplikaci Microsoft Excel. Tento přístup umožňuje důkladně analyzovat a prezentovat získaná data, ačkoliv aplikace použitá pro sběr dat nepodporuje přímé zobrazování závislostí mezi hardwarovými a softwarovými zdroji.

6.7.1 Závislost při záplavovém útoku

SYN flood útok, který je zaměřen na zahlcení serveru falešnými SYN požadavky v rámci TCP three-way handshake, vykazuje minimální využití RAM, ale značné zatížení CPU. Tento útok vyžaduje od serveru zpracování velkého množství nekompletních spojení, což vede k vysokému využití procesorového výkonu kvůli nutnosti reagovat na každý SYN paket. Podrobnější data z měření nabízí graf 6.29, na kterém můžeme vidět jak při nárůstu spojení dojde ke zvýšení procesorové aktivity a v momentě ukončení útoku a postupného rušení spojení k rychlému poklesu.



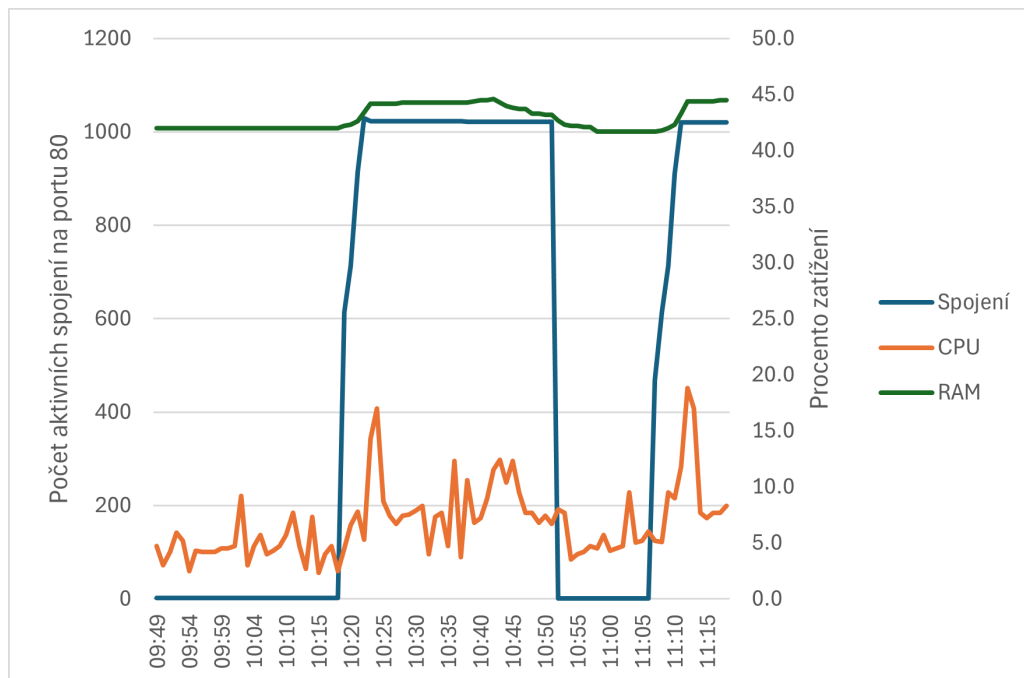
Obr. 6.29: Zobrazení závislosti počtu spojení na vytížení RAM a CPU v průběhu SYN Flood

Z hlediska síťových prostředků, záplavový útok značně zvyšuje množství dat, která protečou síťovou kartou. V případě SYN flood útoku bylo zaznamenáno, že objem dat dosáhl až 19,56 Mb/s, což je výsledek omezeného přenosu mezi virtuálními stroji během útoku.

6.7.2 Závislosti při logickém útoku

Slowloris útok je typem DoS útoku, který cílí na zpomalování webového serveru tím, že drží spojení otevřená po nezvykle dlouhou dobu. Výsledky testu zobrazené

v grafu 6.30 ukazují, že při útoku Slowloris dochází k malému zatížení CPU, ale k výraznému nárůstu využití RAM. Tento nárůst je dán tím, že útok vytváří velké množství polootevřených spojení, která vyžadují paměť pro udržení stavu spojení, ale nevytěžují výpočetní kapacity CPU.



Obr. 6.30: Zobrazení závislosti počtu spojení na vytížení RAM a CPU v průběhu Slowloris útoku

Analýza dat ukazuje, že různé typy DoS útoků mají odlišný dopad na hardwarové a softwarové zdroje serveru. Pochopení těchto rozdílů je nezbytné z hlediska vývoje účinných obranných strategií. Například, zatímco techniky jako geo blokace a rate limiting mohou být účinné proti útokům typu SYN flood, mohou být méně efektivní proti útokům typu Slowloris. Dále také podporuje tvrzení, že není možné jednoznačně určit hraniční veličiny sledovaných parametrů pro DoS útoky obecně.

Závěr

V rámci této bakalářské práce byla vytvořena funkční aplikace, která uživateli umožňuje náhled na dopady útoků na server, byly definovány metody detekce a bylo diskutováno množství stavů a souvislostí s touto tematikou.

Samotná aplikace umožňuje uživatelům získat přehled o různých metrikách, které jsou klíčové pro detekci DoS útoků. Nicméně, během testování se ukázalo, že jedním z omezení aplikace je neschopnost získávat statistiky z Apache serveru během intenzivních DoS útoků. Pokud dojde k přetížení samotného Apache serveru, zastaví se také zobrazování statistik, což snižuje použitelnost aplikace v těchto situacích. Přesto ale zůstává užitečná díky nabídce dalších metrik, které mohou být sledovány.

Dalším aspektem, který by mohl být vylepšen, je způsob přenosu dat mezi skripty a backendem aplikace, a dále mezi backendem a frontendem. Současné řešení používající API může být nahrazeno soketovou komunikací, která by umožnila plynulejší datový tok a zlepšila celkovou efektivitu aplikace.

Díky aplikaci bylo možné analyzovat útoky a navrhnout metody pro jejich detekci. Metody dokáží spolehlivě detekovat většinu testovaných typů útoků, s výjimkou HTTP Flood útoku. Během testování na serveru bez přirozeného provozu byly útoky jasně rozpoznatelné. Nicméně, v reálném provozu by detekce byla složitější a útoky by nebyly tak zřejmé. Uvědomuji si, že pro nasazení v běžném provozu by detekce musela být založena na složitějších principech, například na strojovém učení, které by umožnilo adaptivní rozpoznávání útoků v dynamickém prostředí.

Kromě toho, vytvořená aplikace poskytuje cenné informace, které mohou být použity k vývoji dalších nástrojů a metod pro detekci DoS útoků. Například integrace aplikace s pokročilými analytickými nástroji by mohla výrazně zlepšit schopnost detekovat a reagovat na různé typy útoků. Tento přístup by umožnil lepší využití historických dat a zlepšil by přesnost detekčních algoritmů.

V závěru lze říci, že vytvořená aplikace poskytuje užitečný nástroj pro analýzu a detekci DoS útoků, přičemž nabízí cenné poznatky o chování serveru pod útokem. Přesto existují oblasti, které by mohly být dále vylepšeny, aby aplikace byla robustnější a použitelnější v širším spektru situací. Tato práce tak nejen přináší konkrétní řešení, ale také otevírá prostor pro další výzkum a zdokonalování v oblasti detekce DoS útoků. Výsledky této práce by mohly sloužit jako základ pro budoucí studie a vývoj pokročilých systémů pro ochranu proti DoS útokům, což je v dnešní době neustále rostoucí hrozbou pro internetovou infrastrukturu.

Literatura

- [1] PORTER, Evan. *Co je DDoS útok a jak mu zabránit v roce 2024*. Online. SafetyDetectives. C2024. Dostupné z: <https://cs.safetydetectives.com/blog/co-je-ddos-utok-a-jak-mu-zabranit/>. [cit. 2024-05-19].
- [2] JUREK, Michael. *Detekce moderních Slow DoS útoků*. Online, Diplomová práce, vedoucí Marek Sikora. Brno: Vysoké učení technické v Brně. Fakulta elektrotechniky a komunikačních technologií. Ústav telekomunikací, 2022. Dostupné z: <http://hdl.handle.net/11012/204760>. [cit. 2024-05-20].
- [3] LAKE, Jason. *Differences: TCP/IP vs OSI Model*. Online. OrhanErgun. 2024. Dostupné z: <https://orhanergun.net/tcp-ip-vs-osi-model>. [cit. 2024-05-19].
- [4] *HTTP flood attack*. Online. Cloudflare. C2024. Dostupné z: <https://www.cloudflare.com/learning/ddos/http-flood-ddos-attack/>. [cit. 2024-05-16].
- [5] *HTTP flood attack*. Online. Cloudflare. C2024. Dostupné z: <https://www.cloudflare.com/learning/ddos/http-flood-ddos-attack/>. [cit. 2024-05-20].
- [6] *Intrusion Protection System (IPS) and Intrusion Detection System (IDS)*. Online. SOPHOS. C1997-2024. Dostupné z: <https://www.sophos.com/en-us/cybersecurity-explained/ips-and-ids>. [cit. 2024-05-20].
- [7] BENZL, Lukáš. *Jak rychlé by mělo být načítání webových stránek?* Online. RYCHLOST.cz. 2018. Dostupné z: <https://rychlost.cz/clanek/2017-08-jak-rychle-by-melo-byt-nacitani-webovych-stranek/>. [cit. 2024-05-16].
- [8] *Ping (ICMP) flood DDoS attack*. Online. Cloudflare. United States: Cloudflare, 2022. Dostupné z: <https://www.cloudflare.com/learning/ddos/ping-icmp-flood-ddos-attack/>. [cit. 2022-12-12].
- [9] *Počet DDoS útoků meziročně vzrostl více než čtyřnásobně. Za nárůstem stojí i válka na Ukrajině*. Online. In: Živě. Praha: Živě, 2022. Dostupné z: <https://shorturl.at/da3Yt>. [cit. 2022-12-12].
- [10] *PostgreSQL*. Online. In: Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, 2024. Dostupné z: <https://cs.wikipedia.org/wiki/PostgreSQL>. [cit. 2024-05-16].

- [11] KOĐOUSKOVÁ, Barbora. *Proč k vývoji webových aplikací použít technologii NodeJS?* Online. Rascasone. C2024. Dostupné z: <https://www.rascasone.com/cs/blog/node-js-architektura-moduly-npm>. [cit. 2024-05-16].
- [12] *Python*. Online. Wikisofia. C2013. Dostupné z: <https://wikisofia.cz/wiki/Python>. [cit. 2024-05-16].
- [13] *R U Dead Yet? (R.U.D.Y.) attack*. Online. Cloudflare. C2024. Dostupné z: <https://www.cloudflare.com/learning/ddos/ddos-attack-tools/r-u-dead-yet-rudy/>. [cit. 2024-05-20].
- [14] *Ruští hackeri napadli americká letiště, bezpečnostní úřady vidí spojitost s výbuchem na Kerčském mostu*. Online. In: iRozhlas. Washington: iRozhlas, 2022. Dostupné z: https://www.irozhlas.cz/zpravy-svet/rusko-hackeri-letiste-usa-vybuch-kercsky-most-krym_2210111749_har. [cit. 2022-12-12].
- [15] *Slowloris DDoS attack*. Online. Cloudflare. United States: Cloudflare, 2022. Dostupné z: <https://www.cloudflare.com/learning/ddos/ddos-attack-tools/slowloris/>. [cit. 2022-12-12].
- [16] *Slow Post Attacks*. Online. NETSCOUT. United States: NETSCOUT, 2022. Dostupné z: <https://www.netscout.com/what-is-ddos/slow-post-attacks>. [cit. 2022-12-12].
- [17] *Slow Read DDoS Attacks*. Online. NETSCOUT. NETSCOUT, 2022. Dostupné z: <https://www.netscout.com/what-is-ddos/slow-read-attacks>. [cit. 2022-12-12].
- [18] YASAR, Kinza, NOVOTNY, Amy (ed.). *TCP/IP*. Online. TechTarget. C2000—2024. Dostupné z: <https://www.techtarget.com/searchnetworking/definition/TCP-IP>. [cit. 2024-05-16].
- [19] *TCP SYN Flood*. Online. Imperva. C2024. Dostupné z: <https://www.imperva.com/learn/ddos/syn-flood/>. [cit. 2024-05-20].
- [20] *UDP Flood*. Online. IONOS. C2024. Dostupné z: <https://www.ionos.com/digitalguide/server/security/udp-flood/>. [cit. 2024-05-20].
- [21] DSOUZA, Melisha. *Vulnerabilities in the Application and Transport Layer of the TCP/IP stack*. Online. Pakthub. 2019. Dostupné z: <https://shorturl.at/NFRys>. [cit. 2024-05-20].

- [22] EVANS, Lily. *What Are False Positives In Intrusion Detection Systems*. Online. Storables.com. C2024. Dostupné z: <https://storables.com/home-security-and-surveillance/what-are-false-positives-in-intrusion-detection-systems/>. [cit. 2024-05-20].
- [23] *What is a DDoS Attack?* Online. NETSCOUT. United States: NETSCOUT, 2022. Dostupné z: <https://www.netscout.com/what-is-ddos>. [cit. 2022-12-12].
- [24] *What is an ACK flood DDoS attack?* Online. Cloudflare. C2024. Dostupné z: <https://www.cloudflare.com/learning/ddos/what-is-an-ack-flood/>. [cit. 2024-05-20].
- [25] *What is an intrusion detection and prevention system (IDPS)?* Online. RedHat. 2023. Dostupné z: <https://www.redhat.com/en/topics/security/what-is-an-IDPS>. [cit. 2024-05-20].
- [26] MARQUESES, Rexter. *What Is the Difference Between DoS and DDoS Attacks?* Online. SECURITY GLADIATORS. C2024. Dostupné z: <https://securitygladiators.com/threat/ddos-vs-dos/>. [cit. 2024-05-20].
- [27] *What is the Internet Control Message Protocol (ICMP)?* Online. Cloudflare. United States: Cloudflare, 2022. Dostupné z: <https://www.cloudflare.com/learning/ddos/glossary/internet-control-message-protocol-icmp/>. [cit. 2022-12-12].

Seznam symbolů a zkratek

DoS	Denial of Service
DDoS	Distributed Denial of Service
ICMP	Internet Control Message Protocol
UDP	User Datagram Protocol
TCP	Transmission Control Protocol
TCB	Transmission Control Block
HTTP	Hypertext Transfer Protocol
API	Application Programming Interface
JSON	JavaScript Object Notation
TCP/IP	Transmission Control Protocol/Internet Protocol
OSI	Open Systems Interconnection
IP	Internet Protocol
IDS	Intrusion Detection Systems
IPS	Intrusion Prevention Systems
SSH	Secure Shell
FTP	File Transfer Protocol
NIDS	Network intrusion detection system
HIDS	Host-based intrusion detection system
API	Application Programming Interface
HDD	Hard Disk Drive
VPN	Virtual Private Network

Seznam příloh

A	Obsah elektronické přílohy	77
B	Návod k instalaci vytvořené aplikace	78
B.1	Příprava monitorovacích skriptů	78
B.2	Instalace webové aplikace SMT	78
B.2.1	Instalace Node.js	78
B.2.2	Instalace databáze	78
B.3	Spuštění aplikace	79
B.3.1	Ověření funkčnosti	79
B.4	Propojení scriptů s aplikací	79

A Obsah elektronické přílohy

system-monitoring-tool-final.....	praktická část práce
├─ readme.md	
├─ network_script.....	složka pro monitoring sítě
│ └─ network_script.py	
│ └─ readme.md.....	návod k nastavení skriptu pro síť
├─ server-side_script.....	složka pro monitoring serveru
│ └─ readme.md.....	návod k nastavení skriptu pro server
│ └─ server_script.py	
├─ web-app.....	složka webové aplikace
│ └─ package.json	
│ └─ readme.md.....	návod k instalaci a nastavení webové aplikace a databáze
│ └─ server.js	
│ └─ public.....	veřejná složka webové aplikace
│ └─ favicon.png	
│ └─ css	
│ └─ style.css	
│ └─ style.css.map	
│ └─ js	
│ └─ dashboard.js	
│ └─ mainFE.js	
│ └─ manageConnection.js	
│ └─ routers	
│ └─ api	
│ └─ apiRouter.js	
│ └─ web	
│ └─ webRouter.js	
│ └─ settings	
│ └─ settings.json	
│ └─ utils	
│ └─ dbConnect.js	
│ └─ idGenerator.js	
│ └─ views	
│ └─ err.ejs	
│ └─ manageConnection.ejs	
│ └─ serverDetail.ejs	
│ └─ serverTable.ejs	
│ └─ settings.ejs	
│ └─ components	
│ └─ footer.ejs	
testovani.....	podklady k testování a výsledky
├─ prikazy-pro-testovani.pdf.....	příkazy jednotlivých útoků
├─ grafy.....	export všech grafů
└─ konfiguracni-soubory.....	konfigurační soubory pro útoky

B Návod k instalaci vytvořené aplikace

B.1 Příprava monitorovacích skriptů

Jednotlivé skripty umístíme na požadované umístění a provedem jejich inicializační spuštění.

```
python3 NAZEV_SKRIPTU.py
```

Tímto dojde k vytvoření konfiguračního souboru. Jednotlivé proměnné souboru popisuje kapitola 5. Po zadání správných hodnot můžeme skript znovu spustit.

B.2 Instalace webové aplikace SMT

Pro webovou aplikaci budeme potřebovat Node.js a PostgreSQL.

Poznámka: Návod platný pro operační systémy Linux Ubuntu.

B.2.1 Instalace Node.js

V první řadě začneme stažením repozitáře, nebo extrakcí zip archivu se soubory. Následně nainstalujeme samotný balíček běhového prostředí:

```
sudo apt update
sudo apt install nodejs
sudo apt install npm
```

Nyní přejdeme do adresáře s webovou aplikací, kde je potřeba nainstalovat všechny použité knihovny:

```
npm install
```

Posledním krokem ke spuštění webové aplikace je instalace databáze.

B.2.2 Instalace databáze

Databázi na Linuxové operační systémy můžeme nainstalovat pomocí:

```
sudo apt install postgresql
```

SQL příkazy pro vytvoření databáze a tabulek

Připojení k databázi pomocí nástroje psql příkazové řádky:

```
sudo -u postgres psql
```

Vytvoření uživatele a databáze:

```
CREATE USER smt WITH PASSWORD 'aaa';  
ALTER ROLE smt LOGIN CREATEDB;  
CREATE DATABASE smtdb;  
GRANT ALL PRIVILEGES ON DATABASE smtdb TO smt;
```

Připojení k nové databázi:

```
\c smtdb
```

Kód pro vytvoření schématu a tabulek je v souboru `\web-app\readme.md`

B.3 Spuštění aplikace

Pokud máme nainstalovanou aplikaci a připravenou databázi, můžeme spustit webové rozhraní:

```
npm start
```

Zda je aplikace správně nainstalovaná, se můžeme přesvědčit pomocí webového prohlížeče, do kterého zadáme adresu `http://localhost:3000`.

B.3.1 Ověření funkčnosti

Pokud je vše správně, můžeme vidět stránku s prázdnou tabulkou ‘Data from all servers’. Pokud však zde vidíme error hlášku, je zřejmě chybně připravená databáze. Více informací se dozvíme z terminálu webového serveru aplikace.

B.4 Propojení scriptů s aplikací

Abychom mohli získávat data ze serveru, je potřeba vygenerovat token a připojit skripty.

Vytvoření tokenu provedeme přes stránku ‘Manage script connection’, kde zvolíme možnost ‘Create new connection’ a v načteném formuláři vyplníme požadované údaje. Pole ‘Server name’ a ‘Description’ nemají na funkci aplikace žádný vliv, slouží pouze pro lepší identifikaci přichozích dat. Pole ‘Server IP’ je však důležité pro testování doby odezvy serveru. Do tohoto pole zadáváme IP adresu stroje a rozhraní, na které se provádí útok. Adresa se zadává ve formátu ‘192.168.1.91’. Po odeslání formuláře nám aplikace vrátí 6místný identifikátor serveru, který bude potřeba vložit do konfigurace skriptu.

Poznámka: Konfigurace skriptů je v jejich složkách.